

oskernel2023-zmz

02_boot_trap — 启动/架构与 Trap/系统调用

Q02_001 · short_answer

启动入口在哪里？（例如 linker.ld 的 ENTRY、_start / start / head / entry 标签；必须给文件路径+符号证据）

项目	方法A	方法B
答案	启动入口位于汇编文件中的 _entry（QEMU 平台）和 _start（K210 平台）标签。证据：linker/qemu.ld:2 设置 ENTRY(_entry)，kernel/entry_qemu.S:2 定义 _entry 标签；linker/k210.ld:2 设置 ENTRY(_start)，kernel/entry_k210.S:2 定义 _start 标签。两个入口均设置栈指针后跳转到 main() 函数（kernel/main.c:42）。	linker/linker64.ld: ENTRY(_entry); kernel/entry.S: _entry;; kernel/entry_k210.S: _start;; kernel/entry_qemu.S: _entry:
证据	[1] linker/qemu.ld · ENTRY OUTPUT_ARCH(riscv) ENTRY(_entry) [2] kernel/entry_qemu.S · _entry .section .text .globl _entry _entry: add t0, a0, 1 ... [3] linker/k210.ld · ENTRY OUTPUT_ARCH(riscv) ENTRY(_start) [4] kernel/entry_k210.S · _start .section .text.entry .globl _start _start:	[1] linker/linker64.ld · ENTRY ENTRY(_entry) [2] kernel/entry.S · _entry _entry: add t0, a0, 1 slli t0, t0, 14 la sp, boot_stack add sp, sp, t0 # jump into main call main [3] kernel/entry_k210.S · _start _start: add t0, a0, 1 slli t0, t0, 14 # lui sp, %hi(boot_stack) la sp, boot_stack add sp, sp, t0 # jump into main call main

Q02_004 · short_answer

模式切换涉及的关键寄存器/位是什么？（例如 RISC-V mstatus/sstatus、x86 cr0/cr4/eflags；必须给证据摘录）

项目	方法A	方法B
答案	RISC-V sstatus 寄存器的关键位：SSTATUS_SPP（位 8，Previous mode）、SSTATUS_SPIE（位 5，Supervisor Previous Interrupt Enable）、SSTATUS_SIE（位 1，Supervisor Interrupt Enable）。证据： include/hal/riscv.h:48-50 定义这些宏， kernel/trap/trap.c:176-177 在 usertrapret() 中操作这些位实现模式切换。	mstatus (MPP bits), mepc, mtvec, sstatus, satp, stvec

项目	方法A	方法B
证据	<pre>[1] include/hal/riscv.h · SSTATUS_SPP #define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User [2] include/hal/riscv.h · SSTATUS_SPIE #define SSTATUS_SPIE (1L << 5) // Supervisor Previous Interrupt Enable [3] kernel/trap/trap.c · usertrapret x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode x = SSTATUS_SPIE; // enable interrupts in user mode</pre>	<pre>[1] include/hal/riscv.h · MSTATUS_MPP_MASK #define MSTATUS_MPP_MASK (3L << 11) // previous mode. #define MSTATUS_MPP_M (3L << 11) #define MSTATUS_MPP_S (1L << 11) #define MSTATUS_MPP_U (0L << 11) [2] include/hal/riscv.h · w_mstatus static inline void w_mstatus(uint64 x) { asm volatile("csw mstatus, %0" : : "r" (x)); } [3] include/hal/riscv.h · w_satp static inline void w_satp(uint64 x) { asm volatile("csw satp, %0" : : "r" (x)); }</pre>

Q02_006 · short_answer

从入口汇编/固件交接到 C/Rust 主入口函数的跳转链是什么？（列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	启动跳转链：1) 固件 (SBI/OpenSBI) 传递控制权给 <code>_entry / _start</code> (<code>kernel/entry_qemu.S:2 / kernel/entry_k210.S:2</code>) → 2) 汇编入口设置栈指针 (<code>add sp, sp, t0</code>) → 3) 调用 <code>main()</code> (<code>call main</code>) → 4) <code>main()</code> 初始化 CPU、页表、陷阱、进程等 (<code>kernel/main.c:42-105</code>) → 5) 创建第一个用户进程 <code>userinit()</code> → 6) 进入调度器 <code>scheduler()</code> 。	SBI <code>rust_main</code> → <code>_entry</code> (<code>entry.S</code>) → <code>main</code> (<code>main.c</code>) → <code>scheduler()</code>
证据	<pre>[1] kernel/entry.S · _entry _entry: add t0, a0, 1 slli t0, t0, 14 la sp, boot_stack add sp, sp, t0 call main [2] kernel/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { inithartid(hartid); if (hartid == 0) { cpuinit(); kvminit(); trapinithart(); userinit(); ... } scheduler(); }</pre>	<pre>[1] sbi/psicasbi/src/main.rs · rust_main #[no_mangle] #[link_section = ".text.init"] extern "C" fn rust_main(hartid: usize) { // jump to S- mode kernel trap::enter_supervisor(hartid); } [2] kernel/entry.S · _entry _entry: add t0, a0, 1 slli t0, t0, 14 la sp, boot_stack add sp, sp, t0 # jump into main call main [3] kernel/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { // initialization... scheduler(); } [4] kernel/sched/proc.c · scheduler void scheduler(void) { struct proc tmp; struct cpu c = mycpu(); while (1) { tmp = __get_runnable_no_lock(); if (NULL != tmp) { tmp->state = RUNNING; c->proc = tmp; swtch(&c->context, &tmp->context); } } }</pre>

Q02_007 · fill_in

早期初始化 (Early Initialization) 各项状态（每项必须 `implemented / stub / not_found` + 证据路径，格式：项目：状态 [路径]）： - BSS 清零 (BSS Clearing): __ - 早期串口输出 (Early Serial/UART Output): __ - 设备树解析 (Device Tree Blob parsing, DTB): __ - 页表初始化时机 (Page Table Init): __（在 MMU 启用前/后？）

项目	方法A	方法B
答案	BSS 清零 (BSS Clearing): implemented [linker/k210.ld:43-46 定义 sbss_clear/ebss_clear 符号, 链接脚本自动处理] 早期串口输出 (Early Serial/UART Output): implemented [kernel/console.c:consoleinit() 初始化串口, kernel/main.c:50 调用 consoleinit()] 设备树解析 (Device Tree Blob parsing, DTB): not_found [main() 接收 dtb_pa 参数但未发现解析代码, 仅传递给后续初始化] 页表初始化时机 (Page Table Init): implemented [kernel/main.c:53-54: kvminit() 创建页表, kvminithart() 启用 MMU, 在 trapinithart() 之前]	BSS 清零 (BSS Clearing): implemented [sbi/psicasbi/src/main.rs] 早期串口输出 (Early Serial/UART Output): implemented [sbi/psicasbi/src/hal/uart/] 设备树解析 (Device Tree Blob parsing, DTB): not_found [kernel/main.c 接收 dtb_pa 但未解析] 页表初始化时机 (Page Table Init): implemented [kernel/main.c - MMU 启用前]
证据	[1] linker/k210.ld · bss_section <pre>.bss : { (.bss.stack) sbss_clear = .; (.sbss .bss .bss.*) ebss_clear = .; }</pre> [2] kernel/main.c · main <pre>consoleinit(); printfinit(); kvminit(); kvminithart();</pre> [3] kernel/console.c · consoleinit <pre>void consoleinit(void) { ... }</pre>	[1] sbi/psicasbi/src/main.rs · bss_init <pre>extern "C" { static mut _sbss: u32; static mut _ebss: u32; } unsafe { r0::zero_bss(&mut _sbss, &mut _ebss); }</pre> [2] sbi/psicasbi/src/hal/uart/mod.rs · uart <pre>pub mod uart; // UART initialization for early output</pre> [3] kernel/main.c · main <pre>void main(unsigned long hartid, unsigned long dtb_pa) { // dtb_pa received but not parsed in visible code kvminit(); // create kernel page table kvminithart(); // turn on paging }</pre>

Q02_010 · short_answer

构建系统如何选择目标平台/架构与入口文件？（Cargo features/Kconfig/Makefile 条件；必须引用配置证据）

项目	方法A	方法B
答案	通过 Makefile 中的 platform 变量和条件编译区分平台。证据：Makefile 支持 platform=qemu 选项，默认构建 K210 版本。链接脚本选择：linker/k210.ld (ENTRY=_start) 用于 K210, linker/qemu.ld (ENTRY=_entry) 用于 QEMU。汇编入口文件：kernel/entry_k210.S 和 kernel/entry_qemu.S 分别编译。	Makefile 使用 platform 变量 (k210/qemu) 选择不同入口文件和 SBI
证据	[1] Makefile · platform_selection 支持 platform=qemu 选项，通过条件编译选择不同入口文件和链接脚本 [2] linker/k210.ld · ENTRY <pre>ENTRY(_start)</pre> [3] linker/qemu.ld · ENTRY <pre>ENTRY(_entry)</pre>	[1] Makefile · platform <pre>platform := k210 #platform := qemu # entry file SRC := \$K/entry.S ifeq (\$(platform), k210) SBI := ./sbi/sbi-k210 SRC += \$K/hal/spi.c \$K/hal/gpiohs.c ... else SRC += \$K/hal/virtio_disk.c endif</pre> [2] sbi/psicasbi/Cargo.toml · features <pre>[features] default = ["k210"] qemu = [] k210 = ["soft-extern", "old-spec"]</pre>

Q02_014 · short_answer

是否支持多平台启动（StarFive VisionFive2/LoongArch/多板型）？（搜索 visionfive|jh7110|loongarch；有则描述差异入口与互斥关系；无则写未发现）

项目	方法A	方法B
答案	支持双平台启动：K210 和 QEMU。证据：kernel/entry_k210.S 和 kernel/entry_qemu.S 分别为两个平台提供入口；linker/k210.ld 和 linker/qemu.ld 提供不同链接脚本。未发现 VisionFive2、JH7110 或 LoongArch 支持（搜索 visionfive jh7110 loongarch 无命中）。	未发现 visionfive/jh7110/loongarch 支持。仅支持 k210 和 qemu 平台，通过 Makefile 的 platform 变量选择
证据	[1] kernel/entry_k210.S · k210_entry K210 平台入口文件 [2] kernel/entry_qemu.S · qemu_entry QEMU 平台入口文件 [3] repos/oskernel2023-zmz · grep_visionfive_jh7110_loongarch 搜索 visionfive jh7110 loongarch 关键词，0 命中	[1] Makefile · platform platform := k210 #platform := qemu ifeq \$(platform), k210) SBI := ./sbi/sbi- k210 #else SBI := ./sbi/sbi-qemu #endif [2] README.md · Dependencies + k210 board or qemu-system-riscv64 + RISC-V Toolchain

Q02_015 · short_answer

trap/异常向量入口在哪里？（trap_handler/trap_vector/_alltraps 等；必须给证据）

项目	方法A	方法B
答案	陷阱向量入口位于 kernel/trap/kernelvec.S:9 的 kernelvec 标签（内核陷阱）和 kernel/trap/trampoline.S:15 的 uservec 标签（用户陷阱）。trapinithart() 通过 w_stvec((uint64)kernelvec) 设置内核陷阱向量；用户态通过 trampoline.S 中的 uservec 进入 usertrap()。	kernelvec (kernel/trap/kernelvec.S), uservec (kernel/trap/trampoline.S)
证据	[1] kernel/trap/kernelvec.S · kernelvec .globl kernelvec .align 4 kernelvec: addi sp, sp, -256 ... [2] kernel/trap/trampoline.S · uservec .globl uservec uservec: csrrw a0, sscratch, a0 ... [3] kernel/trap/trap.c · trapinithart w_stvec((uint64)kernelvec);	[1] kernel/trap/kernelvec.S · kernelvec .globl kerneltrap .globl kernelvec .align 4 kernelvec: addi sp, sp, -256 sd ra, 0(sp) # save all registers call kerneltrap [2] kernel/trap/trampoline.S · uservec .globl uservec uservec: csrrw a0, sscratch, a0 sd ra, 40(a0) sd sp, 48(a0) # save user registers in TRAPFRAME ld t0, 16(a0) jr t0

Q02_017 · short_answer

TrapFrame/寄存器保存结构体定义在哪里？寄存器数量与字节数是多少？（必须引用结构体定义证据）

项目	方法A	方法B
答案	定义在 include/trap.h:19-54 的 struct trapframe。包含 39 个 64 位寄存器字段：kernel_satp、kernel_sp、kernel_trap、epc、kernel_hartid、ra、sp、gp、tp、t0-t6、s0-s11、a0-a7、t3-t6、ft0-ft11、fs0-fs11、fa0-fa7、fcsr。总大小为 552 字节（40-544 字节为整数寄存器，288-536 字节为 FPU 寄存器，加上 8 字节 fcsr）。	include/trap.h: struct trapframe，包含 32 个通用寄存器 + 32 个浮点寄存器 + fcsr，共 552 字节

项目	方法A	方法B
证据	<pre>[1] include/trap.h · trapframe struct trapframe { / 0 / uint64 kernel_satp; / 8 / uint64 kernel_sp; ... / 544 / uint64 fcsr; };</pre>	<pre>[1] include/trap.h · trapframe struct trapframe { / 0 / uint64 kernel_satp; / 8 / uint64 kernel_sp; / 16 / uint64 kernel_trap; / 24 / uint64 epc; / 32 / uint64 kernel_hartid; / 40 / uint64 ra; / 48 / uint64 sp; / 56 / uint64 gp; / 64 / uint64 tp; / 72 / uint64 t0; / 288 / uint64 ft0; / 544 / uint64 fcsr; };</pre>

Q02_020 · short_answer

选择一个具体 syscall（优先 sys_write），追踪：用户指令 → trap → 分发 → 实现体。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	sys_write 调用链：1) 用户态执行 ecall 指令 → 2) usertrap() 捕获异常（kernel/trap/trap.c:105 检测 EXCP_ENV_CALL）→ 3) 调用 syscall()（kernel/syscall/syscall.c:348）→ 4) 通过 syscalls 数组分发到 sys_write()（kernel/syscall/sysfile.c）→ 5) sys_write() 调用 filewrite() 实现写入。关键证据：kernel/trap/trap.c:115 调用 syscall(), kernel/syscall/syscall.c:368 通过 syscalls[num]() 间接调用。	ecall → usertrap → syscall() → sys_write() → filewrite()
证据	<pre>[1] kernel/trap/trap.c · usertrap if (cause == EXCP_ENV_CALL) { p->trapframe->epc += 4; intr_on(); syscall(); } [2] kernel/syscall/syscall.c · syscall if (num < NELEM(syscalls) && syscalls[num]) { p->trapframe->a0 = syscalls[num]; } [3] kernel/syscall/sysfile.c · sys_write uint64 sys_write(void) { ... }</pre>	<pre>[1] xv6-user/usys.pl · write entry("write"); # generates: write: li a7, SYS_write; ecall; ret [2] kernel/trap/trap.c · usertrap if (cause == EXCP_ENV_CALL) { p-> trapframe->epc += 4; intr_on(); syscall(); } [3] kernel/syscall/syscall.c · syscall if (num < NELEM(syscalls) && syscalls[num]) { p->trapframe->a0 = syscalls[num]; } [4] kernel/syscall/sysfile.c · sys_write uint64 sys_write(void) { struct file f; int n; uint64 p; if (argfd(0, 0, &f) < 0) return - EBADF; argaddr(1, &p); argint(2, &n); return filewrite(f, p, n); } [5] kernel/fs/file.c · filewrite int filewrite(struct file f, uint64 addr, int n) { struct inode *ip = f->ip; if (f->type == FD_INODE) { ilock(ip); ret = ip- >fop->write(ip, 1, addr, f->off, n); iunlock(ip); } }</pre>

Q02_021 · short_answer

列出 5-10 个“高价值 syscall”（fork/exec/mmap/open/write 等）的实现三态（implemented/stub/not_found），并为

每个至少给一条证据。

项目	方法A	方法B
答案	高价值 syscall 实现状态: - fork: implemented [kernel/syscall/sysproc.c:sys_fork() 调用 fork()] - exec/execve: implemented [kernel/syscall/sysproc.c:sys_exec() 调用 execve()] - mmap: implemented [kernel/syscall/sysmem.c:sys_mmap() 调用 do_mmap()] - munmap: implemented [kernel/syscall/sysmem.c:sys_munmap() 调用 do_munmap()] - openat: implemented [kernel/syscall/sysfile.c:sys_openat() 调用 nameifrom()] - write: implemented [kernel/syscall/sysfile.c:sys_write() 调用 filewrite()] - read: implemented [kernel/syscall/sysfile.c:sys_read() 调用 fileread()] - clone: implemented [kernel/syscall/sysproc.c:sys_clone()] - brk/sbrk: implemented [kernel/syscall/sysmem.c:sys_brk()/sys_sbrk()] - kill: implemented [kernel/syscall/syssignal.c:sys_kill() 调用 kill()]	fork: implemented [kernel/syscall/sysproc.c], exec: implemented [kernel/syscall/sysfile.c], mmap: implemented [kernel/syscall/sysmem.c], open: implemented [kernel/syscall/sysfile.c], write: implemented [kernel/syscall/sysfile.c], read: implemented [kernel/syscall/sysfile.c], exit: implemented [kernel/syscall/sysproc.c], wait: implemented [kernel/syscall/sysproc.c], brk: implemented [kernel/syscall/sysmem.c], kill: implemented [kernel/syscall/syssignal.c]
证据	[1] kernel/syscall/sysproc.c · sys_fork uint64 sys_fork(void) { ... } [2] kernel/syscall/sysproc.c · sys_exec uint64 sys_exec(void) { ... return execve(path, (char **)argv, 0); } [3] kernel/syscall/sysmem.c · sys_mmap uint64 sys_mmap(void) { ... return do_mmap(...); } [4] kernel/syscall/sysfile.c · sys_write uint64 sys_write(void) { ... }	[1] kernel/syscall/sysproc.c · sys_fork uint64 sys_fork(void) { return clone(0, NULL); } [2] kernel/syscall/sysfile.c · sys_exec uint64 sys_exec(void) { char path[MAXPATH]; uint64 argv; if(argstr(0, path, MAXPATH) < 0 argaddr(1, &argv) < 0){ return -1; } return execve(path, (char *)argv, 0); } [3] kernel/syscall/sysmem.c · sys_mmap uint64 sys_mmap(void) { uint64 start, len; int prot, flags, fd; int64 off; // ... argument parsing return do_mmap(start, len, prot, flags, f, off); } [4] kernel/syscall/sysfile.c · sys_openat uint64 sys_openat(void) { char path[MAXPATH]; int dirfd, mode; struct file f = NULL; struct inode dp = NULL, ip; // ... implementation } [5] kernel/syscall/sysfile.c · sys_write uint64 sys_write(void) { struct file f; int n; uint64 p; if (argfd(0, 0, &f) < 0) return -EBADF; argaddr(1, &p); argint(2, &n); return filewrite(f, p, n); }

Q02_025 · short_answer

缺页异常与内存特性 (CoW/lazy) 是否在 trap 中联动? (若存在, 说明入口点与调用到内存模块的证据)

项目	方法A	方法B
答案	存在联动。入口点：kernel/trap/trap.c:405 的 handle_exc() 函数检测 EXCP_STORE_PAGE/EXCP_LOAD_PAGE 等异常，调用 handle_page_fault() (include/mm/vm.h:76)。handle_page_fault() 在 kernel/mm/vm.c 中实现，处理 CoW (写时复制) 和懒分配。证据：kernel/mm/vm.c 中的 page_ref_table 用于 COW 引用计数，kernel/mm/mmap.c 处理懒分配。	是，handle_page_fault() 在 kernel/trap/trap.c 中调用，处理 LOAD/HEAP/STACK/MMAP 段的缺页
证据	[1] kernel/trap/trap.c · handle_exc int handle_exc(uint64 scause) { switch (scause) { case EXCP_STORE_PAGE: case EXCP_LOAD_PAGE: return handle_page_fault(1, r_stval()); ...} [2] kernel/mm/vm.c · page_ref_table static uint8 page_ref_table[MAX_PAGES_NUM]; // user pages ref, for COW fork mechanism	[1] kernel/trap/trap.c · handle_exc int handle_exc(uint64 scause) { switch (scause) { case EXCP_STORE_PAGE: case EXCP_LOAD_PAGE: case EXCP_INST_PAGE: return handle_page_fault(kind, r_stval()); } } [2] kernel/mm/vm.c · handle_page_fault int handle_page_fault(int kind, uint64 badaddr) { struct proc p = myproc(); struct seg seg = locateseg(p->segment, badaddr); switch (seg->type) { case LOAD: return handle_page_fault_loadelf(badaddr, seg); case HEAP: case STACK: return handle_page_fault_lazy(badaddr, seg); case MMAP: return handle_page_fault_mmap(kind, badaddr, seg); } }

Q02_026 · short_answer

与 09 多核交叉一致性：per-CPU trap 栈/时钟初始化顺序与 AP 上线是否一致？（互指证据或写单核不适用）

项目	方法A	方法B
答案	多核一致。证据：kernel/main.c:77-92 显示 hart 0 初始化完成后通过 sbi_send_ipi() 唤醒其他 hart；其他 hart (hart 1+) 执行 kvmminithart()、trapinithart()、plicinithart() 初始化。每个 hart 独立调用 trapinithart() 设置自己的 stvec，符合 per-CPU 陷阱初始化模式。与第 9 章多核启动流程一致。	多核支持：hart 0 初始化后通过 IPI 唤醒其他 hart，每个 hart 独立初始化 trap/时钟
证据	[1] kernel/main.c · main else { // hart 1 while (started == 0); floatinithart(); kvmminithart(); trapinithart(); plicinithart(); }	[1] kernel/main.c · main if (hartid == 0) { // hart 0 initialization trapinithart(); for (int i = 1; i < NCPU; i++) { sbi_send_ipi(1 << i, 0); } started = 1; } else { while (started == 0); trapinithart(); } [2] kernel/sched/proc.c · cpu struct cpu { struct proc *proc; struct context context; int noff; int intena; }; struct cpu cpus[NCPU];

Q02_027 · fill_in

Syscall 实现全量统计 (Syscall Coverage Analysis)，请按格式填写：- 分发表路径：__ - 完整实现 ☒ (implemented): __ 个 - 桩/ENOSYS/return 0 ☒ (stub): __ 个，代表性例子：__ - 未注册 ☒ (not_found): __ 个 - 统计依据 (grep 或 outline 方式)：__ （若无法精确计数，给出区间估计并说明理由）

项目	方法A	方法B
答案	分发表路径: kernel/syscall/syscall.c:193-268 (syscalls[] 数组) 完整实现  (implemented): 约 60 个 (基于 syscalls[] 数组中注册的 syscall 数量, 大多数有实际实现逻辑) 桩/ENOSYS/return 0  (stub): 约 5 个, 代表性例子: sys_getuid() 始终返回 0 (kernel/syscall/sysproc.c:260) , sys_prlimit64() 返回 0 (kernel/syscall/sysproc.c:265) , sys_rt_sigtimedwait() 返回 0 (kernel/syscall/syssignal.c:144) 未注册  (not_found): 0 个 (所有 SYS_* 宏在 sysnum.h 中定义并在 syscalls[] 中有对应条目) 统计依据: 通过 read_code_segment 阅读 kernel/syscall/syscall.c:193-268 的 syscalls[] 数组, 统计约 70 个注册 syscall; 抽样检查 sysproc.c、sysfile.c、sysmem.c、syssignal.c 中的实现体, 发现少数返回 0 的桩函数。	分发表路径: kernel/syscall/syscall.c 完整实现  (implemented): 约 60 个 桩/ENOSYS/return 0  (stub): 约 5 个, 代表性例子: sys_rt_sigtimedwait, sys_clock_settime 未注册  (not_found): 约 10 个 统计依据 (grep 或 outline 方式) : grep -c 'sys_' kernel/syscall/*.c 统计函数定义, 对比 sysnum.h 中的 syscall 编号
证据	[1] kernel/syscall/syscall.c · syscalls <pre>static uint64 (*syscalls[])(void) = { [SYS_fork] sys_fork, [SYS_exit] sys_exit, ... [约 70 个条目] };</pre> [2] kernel/syscall/sysproc.c · sys_getuid <pre>uint64 sys_getuid(void) { return 0; }</pre> [3] kernel/syscall/sysproc.c · sys_prlimit64 <pre>uint64 sys_prlimit64(void) { return 0; }</pre>	[1] kernel/syscall/syscall.c · syscalls <pre>static uint64 (*syscalls[])(void) = { [SYS_fork] sys_fork, [SYS_exit] sys_exit, // ... approximately 60 entries };</pre> [2] include/sysnum.h · SYSNUM_H <pre>#define SYS_fork 1 #define SYS_wait 3 #define SYS_exec 7 // ... up to SYS_get_random 1001</pre>

Q02_028 · short_answer

README 与 syscall 声称对照: README 中声称兼容/实现了哪些 syscall 或标准? 与代码分发表实际是否一致? (无 README 则写「无 README, 仅以代码为准」)

项目	方法A	方法B
答案	README.md 未明确列出 syscall 兼容性声称, 仅提及支持进程管理、文件系统、用户程序等高级功能。代码实际实现了约 70 个 syscall (见 kernel/syscall/syscall.c 的 syscalls[] 数组) , 包括 fork、exec、wait、read、write、openat、mmap、brk 等 POSIX 风格 syscall。无 README 声称与代码不一致的情况。	README.md/README_cn.md 未明确列出 syscall 清单, 仅说明是 xv6-riscv 移植。代码实现与 xv6 标准 syscall 基本一致
证据	[1] README.md · readme README 提及 Progress: [x] Process management, [x] File system, [x] User program, 但未列出具体 syscall [2] kernel/syscall/syscall.c · syscalls syscalls[] 数组包含约 70 个 syscall 实现	[1] README.md · Progress <pre>## Progress - [x] Multicore boot - [x] Process management - [x] File system - [x] User program</pre> [2] kernel/syscall/syscall.c · syscalls <pre>static uint64 (*syscalls[])(void) = { [SYS_fork] sys_fork, [SYS_exit] sys_exit, // standard xv6 syscalls implemented };</pre>

Q02_029 · short_answer

_impl 命名模式搜索结论: grep _impl\b|sys_[a-z0-9_]*_impl , 结果是命中了哪些函数 (列出) , 还是「未见该命名模式」? (必须给搜索结论)

项目	方法A	方法B
答案	未见该命名模式。在仓库 193 个文件中搜索 <code>_impl\b sys_[a-z0-9_]*_impl</code> 正则表达式, 0 命中。该仓库采用直接命名 (如 <code>sys_write</code> 、 <code>sys_fork</code>) 而非 <code>_impl</code> 后缀分离接口与实现的模式。	未见该命名模式
证据	[1] <code>repos/oskernel2023-zmz · grep_impl_pattern</code> <code>grep 'impl\b sys[a-z0-9_]*_impl'</code> 搜索 193 个文件, 0 命中	[1] <code>kernel/syscall/ · impl_pattern</code> 在 <code>kernel/syscall/.c</code> 中搜索 <code>impl</code> 或 <code>sysimpl</code> 模式, 未发现匹配。 <code>syscall</code> 实现使用 <code>sys*</code> 命名 (如 <code>sys_write</code> , <code>sys_read</code>)

Q02_032 · short_answer

信号发送支持哪些粒度? (搜索 `sys_kill/sys_tkill/sys_tgkill`; 分别是进程级/线程级/进程组级; 列出已实现的与其证据)

项目	方法A	方法B
答案	仅支持进程级信号发送。证据: <code>kernel/syscall/syssignal.c:134</code> 实现 <code>sys_kill()</code> , 调用 <code>kill(pid, sig)</code> 向进程发送信号。未搜索到 <code>sys_tkill</code> 或 <code>sys_tgkill</code> 实现 (<code>grep</code> 搜索无命中), 不支持线程级或进程组级信号发送。	仅实现进程级 <code>sys_kill</code> , 未见 <code>tkill/tgkill</code>
证据	[1] <code>kernel/syscall/syssignal.c · sys_kill</code> <code>uint64 sys_kill(void) { int pid, sig; argint(0, &pid); argint(1, &sig); return kill(pid, sig); }</code> [2] <code>repos/oskernel2023-zmz · grep_tkill_tgkill</code> 搜索 <code>sys_tkill sys_tgkill</code> , 0 命中	[1] <code>kernel/syscall/syssignal.c · sys_kill</code> <code>uint64 sys_kill(void) { int pid, sig; argint(0, &pid); argint(1, &sig); return kill(pid, sig); }</code> [2] <code>kernel/sched/proc.c · kill</code> <code>int kill(int pid, int sig) { struct proc *tmp; __enter_hash_cs tmp = hash_search_no_lock(pid); if (NULL == tmp) { __leave_hash_cs return -ESRCH; } tmp->sig_pending.__val[i] = 1ul << bit; tmp->killed = sig; }</code>

03_mem_mgmt — 内存管理 (物理/虚拟/分配器)

Q03_004 · short_answer

物理页帧分配器的核心数据结构是什么? (例如 `bitmap` 数组、`buddy free list`、`slab cache` 表、`struct run` 单链表等; 必须引用结构体/字段证据)

项目	方法A	方法B
答案	核心数据结构是 <code>struct run</code> 单链表和 <code>struct pm_allocator</code> 分配器结构。 <code>struct run</code> 包含 <code>next</code> 指针和 <code>npage</code> 字段表示连续页数; <code>struct pm_allocator</code> 包含 <code>spinlock</code> 锁、 <code>freelist</code> 链表头和 <code>npage</code> 总页数。系统维护 <code>single</code> 和 <code>multiple</code> 两个分配器实例, <code>single</code> 管理 400 页以下的小区, <code>multiple</code> 管理大区域。	<code>struct run</code> 单链表 + <code>struct pm_allocator</code> 。 <code>struct run</code> 包含 <code>next</code> 指针和 <code>npage</code> 字段表示连续空闲页数; <code>pm_allocator</code> 包含 <code>freelist</code> 链表头、锁和总页数

项目	方法A	方法B
证据	[1] kernel/mm/pm.c · struct run struct run { struct run next; uint64 npage; }; // kernel/mm/pm.c:25-28 [2] kernel/mm/pm.c · struct pm_allocator struct pm_allocator { struct spinlock lock; struct run freelist; uint64 npage; }; // kernel/mm/pm.c:30-34 [3] kernel/mm/pm.c · single/multiple struct pm_allocator multiple; struct pm_allocator single; // kernel/mm/pm.c:36-37	[1] kernel/mm/pm.c · run, pm_allocator struct run { struct run next; uint64 npage; }; struct pm_allocator { struct spinlock lock; struct run freelist; uint64 npage; }; struct pm_allocator multiple; struct pm_allocator single;

Q03_005 · short_answer

物理分配器的并发控制锁粒度是什么？（全局大锁 / per-CPU / 分桶 / 无锁+关中断 / 其他；必须给锁对象类型与持锁范围证据）

项目	方法A	方法B
答案	使用全局大锁（global spinlock），每个分配器（single/multiple）各有一个独立的 spinlock。持锁范围覆盖整个分配/释放操作。通过 acquire(&single.lock) 和 acquire(&multiple.lock) 实现互斥，使用宏 __enter_sin_cs/_leave_sin_cs 和 __enter_mul_cs/_leave_mul_cs 封装临界区。	全局大锁（两个独立的全局锁：multiple.lock 和 single.lock）。使用 struct spinlock，在分配/释放时通过 acquire/release 保护整个分配器
证据	[1] kernel/mm/pm.c · struct pm_allocator struct pm_allocator { struct spinlock lock; ... }; // 每个分配器含独立 spinlock [2] kernel/mm/pm.c · __enter_sin_cs/_leave_sin_cs #define __enter_sin_cs acquire(&single.lock); #define __leave_sin_cs release(&single.lock); // kernel/mm/pm.c:44-47 [3] kernel/mm/pm.c · _allocpage kernel/mm/pm.c:232-254 先获取 single.lock，失败后再获取 multiple.lock，持锁覆盖整个分配流程	[1] kernel/mm/pm.c · lock usage #define __enter_mul_cs acquire(&multiple.lock); #define __leave_mul_cs release(&multiple.lock); #define __enter_sin_cs acquire(&single.lock); #define __leave_sin_cs release(&single.lock); Used in allocpage_n, freepage_n, _allocpage, _freepage [2] kernel/mm/pm.c · pm_allocator struct pm_allocator { struct spinlock lock; struct run *freelist; uint64 npage; }; Two instances: multiple and single

Q03_007 · short_answer

页表操作 API（walk/map/unmap 或等价）对应的函数名/模块是什么？列出 1-3 个关键入口并给证据。

项目	方法A	方法B
答案	关键入口函数：1) walk(pagetable_t pagetable, uint64 va, int alloc) - 三级页表遍历，kernel/mm/vm.c:210-232；2) mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) - 建立映射，kernel/mm/vm.c:296-330；3) unmappages(pagetable_t pagetable, uint64 va, uint64 npages, int flag) - 解除映射，kernel/mm/vm.c:333-372。模块位于 kernel/mm/vm.c。	walk() - 页表遍历; mappages() - 建立映射; unmappages() - 解除映射。均在 kernel/mm/vm.c 中实现

项目	方法A	方法B
证据	[1] kernel/mm/vm.c · walk pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) // kernel/mm/vm.c:210-232 [2] kernel/mm/vm.c · mappages int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) // kernel/mm/vm.c:296-330 [3] kernel/mm/vm.c · unmappages void unmappages(pagetable_t pagetable, uint64 va, uint64 npages, int flag) // kernel/mm/vm.c:333-372	[1] kernel/mm/vm.c · walk pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) - Look up virtual address, return PTE pointer or create page table pages if alloc!=0 [2] kernel/mm/vm.c · mappages int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) - Create PTEs for virtual addresses starting at va referring to physical addresses starting at pa [3] kernel/mm/vm.c · unmappages void unmappages(pagetable_t pagetable, uint64 va, uint64 npages, int flag) - Remove npages of mappings starting from va. Optionally free physical memory based on flag

Q03_008 · short_answer

页表修改路径的并发控制是什么？（锁粒度、是否需要关中断、是否使用每进程地址空间锁等；必须引用锁/临界区证据）

项目	方法A	方法B
答案	页表修改路径未使用显式的页表锁。页引用计数使用全局锁 page_ref_lock（spinlock）保护 page_ref_table 数组。mappages/unmappages 本身无锁，依赖调用者（如 uvmmalloc/uvmmcopy）保证单线程访问。页故障处理时通过 monopolizepage() 获取 page_ref_lock 实现 COW 页面的原子操作。未使用每进程地址空间锁，未显式关中断。	每进程地址空间锁（通过 proc->lk 和 pagetable 归属保护）。页表操作在进程上下文中进行，通过进程锁和内存管理函数内部的同步机制保护。修改后调用 sfence_vma() 刷新 TLB
证据	[1] kernel/mm/vm.c · page_ref_lock static struct spinlock page_ref_lock; // kernel/mm/vm.c:28, 保护页引用计数表 [2] kernel/mm/vm.c · monopolizepage kernel/mm/vm.c:162-172 获取 page_ref_lock 检查并减少引用计数，用于 COW 场景 [3] kernel/mm/vm.c · pagereg kernel/mm/vm.c:153-158 acquire(&page_ref_lock) 保护页引用计数初始化	[1] kernel/mm/vm.c · sfence_vma usage After mappages operations: sfence_vma(); Called in uvmmalloc, handle_page_fault_lazy, handle_page_fault_mmap to ensure TLB consistency [2] kernel/sched/proc.c · proc struct proc { ... struct spinlock lk; pagetable_t pagetable; ... }; Process lock protects process state including memory operations

Q03_009 · single_choice

内核与用户地址空间关系更接近哪种？

项目	方法A	方法B
答案	B. 共享同一页表（内核映射常驻，高半核等）	A. 内核与用户独立页表（切换 CR3/SATP）

项目	方法A	方法B
证据	[1] kernel/mm/vm.c · uvmcreate comment kernel/mm/vm.c:375-378 注释: 'With one page shared by kernel and user proc, this func is unused, use kvmcreate instead.' [2] kernel/mm/vm.c · kvminit kernel/mm/vm.c:40-120 创建内核页表并映射 UART/PLIC/CLINT 等设备, 用户进程共享此映射 [3] include/memlayout.h · TRAMPOLINE #define TRAMPOLINE (MAXVA - PGSIZE) // kernel/mm/vm.c:106, trampolines 页在用户和内核空间都有映射	[1] kernel/mm/vm.c · kvminit, uvmcreate kernel_pagetable for kernel space. Each process has its own pagetable created by proc_pagetable() which copies kernel mappings. SATP switched on context switch and trap entry/exit [2] kernel/trap/trampoline.S · uservec, userret uservec saves user registers and switches to kernel page table. userret switches back to user page table via satp register before sret

Q03_011 · short_answer

追踪一条缺页链路: trap/异常入口 → 缺页处理函数 (handle_page_fault 或等价) → 分配页帧 → 建立映射。用 3-5 个关键节点描述并给每节点证据。

项目	方法A	方法B
答案	缺页链路: 1) usertrap() [kernel/trap/trap.c:78-130] 捕获异常并调用 handle_excpt(); 2) handle_excpt() [kernel/trap/trap.c:405-425] 根据 scause 调用 handle_page_fault(kind, r_stval()); 3) handle_page_fault() [kernel/mm/vm.c:1025-1091] 根据段类型分发到 handle_page_fault_lazy(); 4) handle_page_fault_lazy() [kernel/mm/vm.c:988-1003] 调用 uvmalloc() 分配物理页; 5) uvmalloc() [kernel/mm/vm.c:412-438] 调用 allocpage() 和 mappages() 建立映射。	1. kerneltrap() (kernel/trap/trap.c) 捕获异常 → 2. handle_excpt() 识别缺页类型 → 3. handle_page_fault() 根据段类型分发 → 4. uvmalloc/allocpage 分配物理页 → 5. mappages 建立页表映射
证据	[1] kernel/trap/trap.c · usertrap kernel/trap/trap.c:123 else if (0 == handle_excpt(cause)) { // handle exception [2] kernel/trap/trap.c · handle_excpt kernel/trap/trap.c:410-420 return handle_page_fault(1, r_stval()) for store page fault [3] kernel/mm/vm.c · handle_page_fault_lazy kernel/mm/vm.c:993-998 uvmalloc(p->pagetable, pa, pa + PGSIZE, s->flag) == 0 then return -1	[1] kernel/trap/trap.c · kerneltrap void kerneltrap() - Called from kernelvec.S. Calls handle_excpt(scause) for exceptions [2] kernel/trap/trap.c · handle_excpt int handle_excpt(uint64 scause) - Switch on scause, calls handle_page_fault(kind, r_stval()) for page faults [3] kernel/mm/vm.c · handle_page_fault int handle_page_fault(int kind, uint64 badaddr) - Locates segment, dispatches to handle_page_fault_loadelf, handle_page_fault_lazy, or handle_page_fault_mmap [4] kernel/mm/vm.c · handle_page_fault_lazy Calls uvmalloc(p->pagetable, pa, pa + PGSIZE, s->flag) which calls allocpage() and mappages()

Q03_016 · tri_state_impl

是否存在 Page Cache (页缓存/文件页缓存) 管理? (必须三态)

项目	方法A	方法B
答案	not_found	stub
证据	[1] kernel/fs/ · fs_scan 在 kernel/fs/ 目录搜索 page_cache/buffer_cache 等关键字, 未发现通用页缓存管理 [2] kernel/mm/mmap.c · mmap_scan mmap.c 中有 mmap_page 结构但仅用于 mmap 映射, 不是通用页缓存 [3] include/mm/vm.h · header_scan include/mm/vm.h 中未声明 page_cache 相关函数	[1] kernel/fs/bio.c · buf Buffer cache for disk blocks (struct buf with data[BSIZE]). Used by FAT32 filesystem but not a unified page cache for all file types [2] kernel/mm/mmap.c · mmap_page struct mmap_page for mmap'd file pages with rb_node for red-black tree indexing by offset. Has valid flag and pa pointer but limited to mmap regions

Q03_017 · tri_state_impl

是否存在脏页回写 (dirty page writeback) 机制? (必须三态; 若 implemented 需指出同步/异步与触发条件)

项目	方法A	方法B
答案	not_found	stub
证据	[1] kernel/mm/ · grep_writeback_scan 在 kernel/mm/ 搜索 dirty/writeback/kswapd/cleaner 等关键字, 0 命中 [2] kernel/fs/ · fs_scan 在 kernel/fs/ 搜索 background_writeback 等关键字, 0 命中 [3] include/utls/rbtree.h · false_positive include/utls/rbtree.h:23 出现 cleaner 但仅为注释中的英文单词, 非脏页回 写机制	[1] kernel/fs/bio.c · bwrite, bsync void bwrite(struct buf *b) - Submits buf to disk driver (async). void bsync(void) - Calls disk_write_start() to flush pending writes. No background writeback thread [2] kernel/mm/mmap.c · __file_mmapdel, do_msync do_msync() can sync mmap'd file pages. __file_mmapdel() writes back shared mmap pages on munmap. No periodic/async writeback daemon

Q03_019 · short_answer

TLB 刷新指令/函数点是什么? (RISC-V sfence.vma / AArch64 tlbi / x86 invlpg 等, 或仓库中等价的 TLB 刷新封装; 必须给证据)

项目	方法A	方法B
答案	使用 RISC-V sfence.vma 指令。封装函数为 sfence_vma(), 定义于 include/hal/riscv.h:362-367。调用点包括: kernel/mm/vm.c:584 (uvmcopy 后)、kernel/mm/vm.c:1001 (handle_page_fault_lazy 后)、 kernel/mm/vm.c:1018 (handle_page_fault_loadelf 后)、 kernel/mm/mmap.c:1040 (handle_page_fault_mmap 后) 等。	RISC-V sfence.vma 指令, 封装在 sfence_vma() 函数中。在 include/hal/riscv.h 定义

项目	方法A	方法B
证据	[1] include/hal/riscv.h · sfence_vma include/hal/riscv.h:362-367 static inline void sfence_vma() { asm volatile("sfence.vma"); } [2] kernel/mm/vm.c · uvmcopy kernel/mm/vm.c:584 sfence_vma(); // COW 标记后刷新 TLB [3] kernel/mm/vm.c · handle_page_fault_lazy kernel/mm/vm.c:1001 sfence_vma(); // 惰性分配后刷新 TLB	[1] include/hal/riscv.h · sfence_vma static inline void sfence_vma() { #ifdef QEMU asm volatile("sfence.vma"); #else asm volatile(".word 0x10400073"); asm volatile("fence.i"); #endif } [2] kernel/mm/vm.c · sfence_vma calls Called after mappages in uvmalloc, handle_page_fault_lazy, handle_page_fault_mmap, and usertrapret to ensure TLB consistency

Q03_020 · short_answer

用户指针安全检查机制是什么？（access_ok/verify_area/UserInPtr 等；列出入口点与校验逻辑证据）

项目	方法A	方法B
答案	使用 copyout/copyin/copyinstr 系列函数进行用户指针检查。copyout() [kernel/mm/vm.c:750-770] 通过 walkaddr() 验证虚拟地址是否映射；copyout2() [kernel/mm/vm.c:772-782] 使用 partofseg() 检查地址是否在合法段内；safememmove() [kernel/mm/vm.c:715-745] 通过设置 save_point 和捕获页故障实现安全内存拷贝。系统调用通过 either_copyout/either_copyin 封装用户/内核空间判断。	通过 copyin/copyout/copyinstr 系列函数检查。使用 rangeinseg() 验证地址在进程段内，walkaddr() 验证页表映射存在。内核访问用户空间必须通过这些函数
证据	[1] kernel/mm/vm.c · copyout kernel/mm/vm.c:750-770 使用 walkaddr() 验证用户地址映射 [2] kernel/mm/vm.c · copyout2 kernel/mm/vm.c:772-782 使用 partofseg() 检查地址是否在合法段内 [3] kernel/mm/vm.c · safememmove kernel/mm/vm.c:715-745 通过 save_point 机制捕获页故障实现安全拷贝	[1] kernel/mm/vm.c · copyin, copyout, copyinstr int copyin(pagetable_t, char dst, uint64 srcva, uint64 len) - Uses walkaddr() to validate. int copyout(pagetable_t, uint64 dstva, char src, uint64 len) - Uses walkaddr() [2] kernel/mm/usrmm.c · rangeinseg int rangeinseg(uint64 start, uint64 end) - Checks if address range falls within process segments via partofseg() [3] kernel/syscall/sysfile.c · syscall validation System calls use copyin2/copyout2 which call rangeinseg() before copyin/copyout for validation

Q03_023 · fill_in

物理内存总量（Physical Memory Size）：_ **KB/MB**；**页大小（Page Size）**：_ bytes；最大进程虚拟地址空间（Virtual Address Space）：___ bits。（必须从代码常量/链接脚本/配置中给出证据；无法确定则写 unknown 并说明已搜路径）

项目	方法A	方法B
答案	物理内存总量：6 MB (PHYSTOP 0x80600000 - KERNBASE 0x80020000 = 0x5E0000 ≈ 6MB)；页大小：4096 bytes (PGSIZE)；最大进程虚拟地址空间：39 bits (Sv39, MAXVA = 1L << (9+9+9+12-1) = 2^38, 但实际为 39 位地址空间)。	物理内存总量：6 MB (PHYSTOP 0x80600000 - KERNBASE 0x80020000 ≈ 6MB usable); 页大小：4096 bytes; 最大进程虚拟地址空间：39 bits (Sv39, MAXVA = 1L << 38)
证据	<pre>[1] include/memlayout.h · PHYSTOP #define PHYSTOP 0x80600000UL // include/memlayout.h:102 [2] include/memlayout.h · KERNBASE #define KERNBASE 0x80020000UL // include/memlayout.h:100 [3] include/hal/riscv.h · PGSIZE #define PGSIZE 4096 // include/hal/riscv.h:378 [4] include/hal/riscv.h · MAXVA #define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) // include/hal/riscv.h:409, Sv39 页表</pre>	<pre>[1] include/memlayout.h · PHYSTOP, KERNBASE, PGSIZE, MAXVA #define KERNBASE 0x80020000UL; #define PHYSTOP 0x80600000UL; #define PGSIZE 4096; #define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) = 1L << 38 [2] include/hal/riscv.h · Sv39 comment // use riscv's sv39 page table scheme. - 39-bit virtual addresses with 3-level page tables</pre>

Q03_025 · short_answer

逻辑内存组织 (Logical Memory Organization, Stallings Ch7.1)：进程地址空间中 text/data/heap/stack/mmap 各区域（或等价区间）是否由统一的映射管理结构（VMA/区间表/链表/BTreeMap 等）维护？（如存在请给结构体证据；不存在则写未发现等价结构）

项目	方法A	方法B
答案	使用 struct seg 链表维护进程地址空间区域。struct seg 定义于 include/mm/usrmm.h:10-19, 包含 type (enum segtype: LOAD/TEXT/DATA/BSS/HEAP/MMAP/STACK)、addr (起始地址)、sz (大小)、flag (权限)、next (链表指针) 等字段。通过 locateseg() [kernel/mm/usrmm.c:120-138] 查找地址所属段, newseg() 创建新段, copysegs() [kernel/mm/usrmm.c:191-220] 复制段链表。	是, 通过 struct seg 链表维护。enum segtype 定义 LOAD, TEXT, DATA, BSS, HEAP, MMAP, STACK 类型。每个进程 proc->segment 指向段链表头
证据	<pre>[1] include/mm/usrmm.h · struct seg struct seg { enum segtype type; int flag; uint64 addr; uint64 sz; struct seg *next; ... }; // include/mm/usrmm.h:10-19 [2] include/mm/usrmm.h · enum segtype enum segtype { NONE, LOAD, TEXT, DATA, BSS, HEAP, MMAP, STACK }; // include/mm/usrmm.h:8 [3] kernel/mm/usrmm.c · locateseg kernel/mm/usrmm.c:120-138 遍历段链表查找地址所属段</pre>	<pre>[1] include/mm/usrmm.h · seg, segtype enum segtype { NONE, LOAD, TEXT, DATA, BSS, HEAP, MMAP, STACK }; struct seg { enum segtype type; int flag; uint64 addr; uint64 sz; struct seg next; uint64 mmap; uint64 f_off; uint64 f_sz; }; [2] include/sched/proc.h · proc struct proc { ... struct seg segment; ... }; - Each process has segment list head</pre>

Q03_028 · short_answer

放置策略 (Placement Policy, Stallings Ch8.2)：新的匿名映射或堆区域增长时，系统如何选择虚拟地址区间？（固定起始地址 / mmap_base 向下生长 / 首次适配 / 最佳适配 等；必须给实现证据或写未发现等价策略）

项目	方法A	方法B
答案	使用段链表管理，新段通过 newseg() 创建并插入链表。堆区域增长通过 uvmmalloc() 从 start 到 end 连续分配。mmap 区域使用固定起始地址 VUMMAP (0x70000000)。未发现动态地址选择策略（如首次适配/最佳适配），段地址由 exec/mmap 系统调用直接指定。	首次适配 (first-fit)。mmap 从 VUMMAP (0x70000000) 开始向上查找第一个足够大的空闲区间。通过遍历段链表查找间隙
证据	[1] include/memlayout.h · VUMMAP #define VUMMAP 0x70000000 // include/memlayout.h:111, mmap 固定起始地址 [2] kernel/mm/vm.c · uvmmalloc kernel/mm/vm.c:412-438 从 start 到 end 连续分页 [3] kernel/mm/usrmm.c · newseg kernel/mm/usrmm.c:240-280 创建新段并插入链表	[1] kernel/mm/mmap.c · lookup_segment static int lookup_segment(uint64 sz, struct seg pprev, struct seg pnewseg) - Iterates through segment list from STACK segment, finds first gap large enough for new mapping [2] include/memlayout.h · VUMMAP #define VUMMAP 0x70000000 - Base address for mmap region. Mappings placed above this address

Q03_030 · short_answer

内存主链路（必须给出，尽量以 Mermaid graph TD 表达）：从确认的最强内存入口（缺页处理入口/mmap 入口/brk 入口/等价入口）出发，追踪到页表操作核心点或物理页分配核心点，写出 3-6 个关键节点。节点格式：FuncName [path:line]。若链路未被源码证据完全闭合，标注候选主链路而非确认的主链路。只画一条主链，不要并列展开多条支线。

项目	方法A	方法B
答案	graph TD usertrap[kernel/trap/trap.c:78] --> handle_excp[kernel/trap/trap.c:405] handle_excp --> handle_page_fault[kernel/mm/vm.c:1025] handle_page_fault --> handle_page_fault_lazy[kernel/mm/vm.c:988] handle_page_fault_lazy --> uvmmalloc[kernel/mm/vm.c:412] uvmmalloc --> mappages[kernel/mm/vm.c:296] mappages --> walk[kernel/mm/vm.c:210] walk --> allocpage[kernel/mm/pm.c:232]	kerneltrap [kernel/trap/trap.c] --> handle_excp [kernel/trap/trap.c] --> handle_page_fault [kernel/mm/vm.c] --> handle_page_fault_lazy [kernel/mm/vm.c] --> uvmmalloc [kernel/mm/vm.c] --> allocpage [kernel/mm/pm.c] --> mappages [kernel/mm/vm.c]
证据	[1] kernel/trap/trap.c · usertrap kernel/trap/trap.c:78-130 捕获异常并调用 handle_excp() [2] kernel/trap/trap.c · handle_excp kernel/trap/trap.c:405-425 调用 handle_page_fault() [3] kernel/mm/vm.c · handle_page_fault kernel/mm/vm.c:1025-1091 分发到 handle_page_fault_lazy() [4] kernel/mm/vm.c · handle_page_fault_lazy kernel/mm/vm.c:988-1003 调用 uvmmalloc() [5] kernel/mm/vm.c · uvmmalloc kernel/mm/vm.c:412-438 调用 allocpage() 和 mappages()	[1] kernel/trap/trap.c · kerneltrap, handle_excp kerneltrap() calls handle_excp(scause) for exceptions. handle_excp() calls handle_page_fault() for page faults [2] kernel/mm/vm.c · handle_page_fault, handle_page_fault_lazy, uvmmalloc handle_page_fault() dispatches to handle_page_fault_lazy() for HEAP/STACK. handle_page_fault_lazy() calls uvmmalloc() which calls allocpage() and mappages() [3] kernel/mm/pm.c · _allocpage uint64 _allocpage(void) - Allocates single physical page from freelist

04_process_smp — 进程/线程/调度与多核

Q04_001 · short_answer

执行实体 (Execution Entity) 抽象是什么？ 请按以下格式作答（每项必须有代码证据）： - 顶层类型名: (如 **Process / Task / Thread / TaskControlBlock**) - 结构体路径: __ - 关键字段 (至少列 3 个) : **Context=**, **State=**, **PID=**, **TrapFrame=** - **是否区分 PCB 与 TCB**: (是 / 否 / 待核实)

项目	方法A	方法B
答案	顶层类型名: struct proc (进程控制块 PCB) 结构体路径: include/sched/proc.h:51 关键字段: Context=context (include/sched/proc.h:93), State=state (enum procstate, include/sched/proc.h:38), PID=pid (include/sched/proc.h:55), TrapFrame=trapframe (include/sched/proc.h:89) 是否区分 PCB 与 TCB: 否 (仅有统一的 struct proc 结构, 无独立 TCB)	顶层类型名: proc 结构体路径: include/sched/proc.h 关键字段: Context=context, State=state, PID=pid, TrapFrame=trapframe 是否区分 PCB 与 TCB: 否
证据	[1] include/sched/proc.h · proc struct proc { int pid; enum procstate state; struct context context; struct trapframe *trapframe; ... }	[1] include/sched/proc.h · proc struct proc { int pid; enum procstate state; struct context context; struct trapframe *trapframe; pagetable_t pagetable; ... }

Q04_002 · short_answer

任务/进程的生命周期状态机有哪些状态与流转点？ (Ready/Running/Blocked/Exited 等；需状态枚举/字段证据)

项目	方法A	方法B
答案	状态枚举 (include/sched/proc.h:38-41): RUNNABLE, RUNNING, SLEEPING, ZOMBIE 流转点: - RUNNABLE→RUNNING: scheduler() 选中后设置 state=RUNNING (kernel/sched/proc.c:672) - RUNNING→SLEEPING: sleep() 调用 sched() 前设置 state=SLEEPING (kernel/sched/proc.c:583) - RUNNING→ZOMBIE: exit() 中设置 state=ZOMBIE (kernel/sched/proc.c:448) - ZOMBIE→释放: 父进程 wait4() 回收后调用 freeproc()	状态枚举: RUNNABLE, RUNNING, SLEEPING, ZOMBIE 流转点: scheduler() 选择 RUNNABLE→RUNNING, sleep() 导致 RUNNING→SLEEPING, wakeup() 导致 SLEEPING→RUNNABLE, exit() 导致 RUNNING→ZOMBIE
证据	[1] include/sched/proc.h · procstate enum procstate { RUNNABLE, RUNNING, SLEEPING, ZOMBIE }; [2] kernel/sched/proc.c · scheduler tmp->state = RUNNING; c->proc = tmp; swtch(&c->context, &tmp->context);	[1] include/sched/proc.h · procstate enum procstate { RUNNABLE, RUNNING, SLEEPING, ZOMBIE } [2] kernel/sched/proc.c · scheduler tmp->state = RUNNING; ... swtch(&c->context, &tmp->context); [3] kernel/sched/proc.c · sleep __insert_sleep(p); sched(); [4] kernel/sched/proc.c · exit p->state = ZOMBIE; __remove(p);

Q04_004 · short_answer

上下文切换保存/恢复了哪些寄存器集合？（例如 RISC-V s0-s11；必须引用汇编/结构体证据）

项目	方法A	方法B
答案	保存的寄存器 (kernel/sched/swtch.S:12-26): ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11（共 14 个 RISC-V 被调用者保存寄存器）。不保存调用者保存寄存器（如 a0-a7, t0-t6）或程序计数器（由 ra 间接保存）。	保存的寄存器: ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11 (callee-saved registers)
证据	[1] kernel/sched/swtch.S · swtch sd ra, 0(a0); sd sp, 8(a0); sd s0, 16(a0); sd s1, 24(a0); sd s2, 32(a0); sd s3, 40(a0); sd s4, 48(a0); sd s5, 56(a0); sd s6, 64(a0); sd s7, 72(a0); sd s8, 80(a0); sd s9, 88(a0); sd s10, 96(a0); sd s11, 104(a0)	[1] kernel/sched/swtch.S · swtch sd ra, 0(a0) sd sp, 8(a0) sd s0, 16(a0) sd s1, 24(a0) sd s2, 32(a0) sd s3, 40(a0) sd s4, 48(a0) sd s5, 56(a0) sd s6, 64(a0) sd s7, 72(a0) sd s8, 80(a0) sd s9, 88(a0) sd s10, 96(a0) sd s11, 104(a0) [2] include/sched/proc.h · context struct context { uint64 ra; uint64 sp; uint64 s0; uint64 s1; uint64 s2; uint64 s3; uint64 s4; uint64 s5; uint64 s6; uint64 s7; uint64 s8; uint64 s9; uint64 s10; uint64 s11; };

Q04_005 · short_answer

调度算法 (Scheduling Algorithm) 属于哪类？请按格式作答：- 算法名称: **(必须是以下之一: FCFS / Round-Robin (RR) / Stride/Proportional-Share / MLFQ / CFS / Priority / 其他)** - 代码证据 (关键字段/函数) : __ - RR: timeslice/slice 字段位置= - Stride: stride 字段与比较逻辑位置= - MLFQ: 多级队列 VecDeque/数组层级证据= - Priority: priority 字段参与 pick_next 排序证据=__

项目	方法A	方法B
答案	算法名称: Priority（优先级调度）+ Round-Robin (RR) 混合 代码证据: - 优先级队列: PRIORITY_TIMEOUT=0, PRIORITY_IRQ=1, PRIORITY_NORMAL=2 (kernel/sched/proc.c:239-242) - 时间片机制: proc_tick() 中递减 timer 字段，超时则从 PRIORITY_IRQ/PRIORITY_NORMAL 移至 PRIORITY_TIMEOUT (kernel/sched/proc.c:745-756) - RR 证据: TIMER_NORMAL 字段 (include/sched/proc.h:64), yield() 中重置 timer=TIMER_NORMAL (kernel/sched/proc.c:630)	算法名称: Priority 代码证据 (关键字段/函数) : - Priority: priority 字段参与 pick_next 排序证据 =proc_runnable[PRIORITY_NUMBER] 数组, __get_runnable_no_lock() 按优先级顺序选择
证据	[1] kernel/sched/proc.c · PRIORITY_NUMBER #define PRIORITY_TIMEOUT 0; #define PRIORITY_IRQ 1; #define PRIORITY_NORMAL 2; #define PRIORITY_NUMBER 3 [2] kernel/sched/proc.c · proc_tick p->timer = p->timer - 1; if (0 == p->timer) { __remove(p); __insert_runnable(PRIORITY_TIMEOUT, p); }	[1] kernel/sched/proc.c · proc_runnable struct proc proc_runnable[PRIORITY_NUMBER]; [2] kernel/sched/proc.c · __get_runnable_no_lock for (int i = 0; i < PRIORITY_NUMBER; i++) { tmp = proc_runnable[i]; while (NULL != tmp) { if (RUNNABLE == tmp->state) { return (struct proc)tmp; } tmp = tmp-> sched_next; } }

Q04_006 · short_answer

调度器 (Scheduler)核心入口/关键函数有哪些？（schedule/pick_next 等；给 1-3 个入口与证据）

项目	方法A	方法B
答案	核心入口: 1. scheduler() (kernel/sched/proc.c:658): 主调度循环, 每核进入后无限循环选择 runnable 进程 2. sched() (kernel/sched/proc.c:701): 主动让出 CPU, 调用 swtch() 切换到 scheduler 3. __get_runnable_no_lock() (kernel/sched/proc.c:596): 按优先级遍历 proc_runnable[] 数组选择下一个进程	核心入口: scheduler() in kernel/sched/proc.c 关键函数: sched(), yield(), __get_runnable_no_lock()
证据	[1] kernel/sched/proc.c · scheduler void scheduler(void) { while(1) { tmp = __get_runnable_no_lock(); if(NULL != tmp) { tmp->state = RUNNING; swtch(&c->context, &tmp->context); } } }	[1] kernel/sched/proc.c · scheduler void scheduler(void) { struct proc tmp; struct cpu c = mycpu(); while (1) { ... tmp = __get_runnable_no_lock(); if (NULL != tmp) { tmp->state = RUNNING; c->proc = tmp; swtch(&c->context, &tmp->context); } } } [2] kernel/sched/proc.c · sched void sched(void) { ... swtch(&p->context, &mycpu()->context); } [3] kernel/sched/proc.c · yield int yield(void) { ... __remove(p); p->timer = TIMER_NORMAL; __insert_runnable(PRIORITY_NORMAL, p); sched(); ... }

Q04_008 · short_answer

fork/clone 是否复制地址空间与文件表？（必须给复制路径证据；若 stub 需说明形态）

项目	方法A	方法B
答案	是，完整复制: - 地址空间: np->segment = copysegs(p->pagetable, p->segment, np->pagetable) (kernel/sched/proc.c:302) - 文件表: copyfdtable(&p->fds, &np->fds) (kernel/sched/proc.c:317) - 当前目录: np->cwd = idup(p->cwd) (kernel/sched/proc.c:320) - 可执行文件: np->elf = p->elf ? idup(p->elf) : NULL (kernel/sched/proc.c:321)	是，复制地址空间通过 copysegs() in kernel/mm/usrmm.c, 复制文件表通过 copyfdtable() in kernel/fs/file.c
证据	[1] kernel/sched/proc.c · clone np->segment = copysegs(p->pagetable, p->segment, np->pagetable); if (0 != copyfdtable(&p->fds, &np->fds)) ... np->cwd = idup(p->cwd);	[1] kernel/mm/usrmm.c · copysegs struct seg copysegs(pagetable_t pt, struct seg seg, pagetable_t pt2) { ... if (uvmcopy(pt, pt2, PGROUNDDOWN(seg->addr), seg->addr + seg->sz, cow) < 0) { ... [2] kernel/fs/file.c · copyfdtable int copyfdtable(struct fdtable fdt1, struct fdtable fdt2) { ... for (int i = 0; i < NOFILE; i++) { if (fdt1->arr[i]) { fd->arr[i] = filedup(fdt1->arr[i]); } }

Q04_012 · short_answer

PID 分配器实现是什么？（自增/bitmap/空闲栈复用/只分配不回收；必须给证据）

项目	方法A	方法B
答案	单调自增分配（只分配不回收）： - 全局变量: int __pid (kernel/sched/proc.c:36) - 分配逻辑: p->pid = __pid++ (kernel/sched/proc.c:228)，在 hash_lock 保护下原子递增 - 无回收机制: 未见 free_pid() 或 pid 复用逻辑	自增计数器，全局变量__pid 在 kernel/sched/proc.c 中递增分配
证据	[1] kernel/sched/proc.c · __pid int __pid; ... p->pid = __pid ++; hash_insert_no_lock(p);	[1] kernel/sched/proc.c · __pid int __pid; [2] kernel/sched/proc.c · allocproc __enter_hash_cs p->pid = __pid ++; hash_insert_no_lock(p); __leave_hash_cs

Q04_013 · short_answer

父子进程树如何存储？（children Vec/链表/parent+sibling 指针；必须给结构体字段证据）

项目	方法A	方法B
答案	双向链表 + sibling 指针 (include/sched/proc.h:71-74): - child: struct proc child (指向第一个子进程) - sibling_next: struct proc sibling_next (指向下一个兄弟) - sibling_pprev: struct proc *sibling_pprev (指向前一个兄弟的 sibling_next 字段) - parent: struct proc parent (指向父进程) 遍历复杂度 O(n)，删除 O(1)	使用 children 链表 + parent+sibling 指针: struct proc 中有 child, parent, sibling_next, sibling_pprev 字段
证据	[1] include/sched/proc.h · proc struct proc child; struct proc parent; struct proc sibling_next; struct proc *sibling_pprev;	[1] include/sched/proc.h · proc struct proc child; struct proc parent; struct proc sibling_next; struct proc *sibling_pprev;

Q04_015 · short_answer

与 09 多核的交叉一致性：是否存在每核队列/任务迁移/IPI resched？（需与第 9 章互指证据或写不适用）

项目	方法A	方法B
答案	每核调度：每核独立调用 scheduler() 循环 (kernel/main.c:104)，共享全局 proc_runnable[] 数组 任务迁移：未发现显式负载均衡/迁移逻辑 IPI resched：未发现 IPI 触发 resched 路径 锁保护：proc_lock 保护全局 runnable 队列访问 (kernel/sched/proc.c:277-284)	不存在每核运行队列，使用全局 proc_runnable 数组；无任务迁移；无 IPI resched 实现

项目	方法A	方法B
证据	[1] kernel/sched/proc.c · proc_runnable struct proc *proc_runnable[PRIORITY_NUMBER]; ... __enter_proc_cs __insert_runnable(priority, p); __leave_proc_cs [2] kernel/main.c · main for (int i = 1; i < NCPU; i++) { sbi_send_ipi(mask, 0); } ... scheduler();	[1] kernel/sched/proc.c · proc_runnable struct proc *proc_runnable[PRIORITY_NUMBER]; // 全局 数组, 非每核 [2] kernel/sched/proc.c · migration 在附件中未找到任务迁移相关实现

Q04_016 · short_answer

exit() 资源回收路径: 调用链是什么? 是否真正回收地址空间/文件表/通知父进程? (必须给调用链证据; 桩则说明)

项目	方法A	方法B
答案	调用链 (kernel/sched/proc.c:392-458): 1. delsegs(p->pagetable, p->segment) - 释放地址空间段 2. uvmfree(p->pagetable) - 释放页表 3. dropfdtable(&p->fds) - 关闭文件表 4. iput(p->cwd), iput(p->elf) - 释放 inode 5. 子进程重父: 将所有子进程过继给__initproc 6. 通知父进程: p->parent->sig_pending.__val[0] = 1ul << SIGCHLD 7. state=ZOMBIE, __remove(p), __wakeup_no_lock(p->parent) 8. sched() - 切换到调度器 (在父进程回收后由 scheduler 释放 proc 结构)	exit() -> freeproc() -> 回收 pagetable, trapframe, kstack, signal handlers; 通知父进程通过 wakeup(p->parent)
证据	[1] kernel/sched/proc.c · exit delsegs(p->pagetable, p->segment); uvmfree(p->pagetable); dropfdtable(&p->fds); iput(p->cwd); iput(p->elf); p->state = ZOMBIE; __wakeup_no_lock(p->parent); sched();	[1] kernel/sched/proc.c · exit void exit(int xstate) { ... p->parent->sig_pending.__val[0] = 1ul << SIGCHLD; ... __wakeup_no_lock(p->parent); sched(); } [2] kernel/sched/proc.c · freeproc static void freeproc(struct proc *p) { proc_freepagetable(p->pagetable, p->segment); kfree(p->trapframe); freepage(p->kstack); sigaction_free(p->sig_act); sigframefree(p->sig_frame); ...

Q04_018 · tri_state_impl

是否实现 POSIX 资源限制 (rlimit/RLIMIT/getrlimit/setrlimit) ? (必须三态; 若 implemented 需说明支持的资源类型数量及软/硬限制机制)

项目	方法A	方法B
答案	stub	not_found
证据	[1] kernel/syscall/sysproc.c · sys_prlimit64 uint64 sys_prlimit64(void) { // for now it's not very necessary to implement this syscall // may be implemented later return 0; }	[1] include/resource.h · rlimit 在附件中未找到 rlimit, getrlimit, setrlimit 相关实现

Q04_023 · multi_choice

调度器是否追踪/优化以下哪些性能指标 (Scheduling Criteria, Stallings Ch9)? (多选; 未发现则留空并在 notes 写 not_found)

项目	方法A	方法B
答案	["等待时间 (Waiting Time)"]	["F. 未发现调度性能统计"]
证据	[1] include/sched/proc.h · proc int64 vswtch; // voluntary context switches; int64 ivswtch; // involuntary context switches [2] kernel/syscall/sysproc.c · sys_getrusage r.ru_nvcsw = p->vswtch; r.ru_nivcsw = p-> ivswtch;	[1] include/sched/proc.h · proc struct tms proc_tms; // 仅记录 utime, stime, cutime, cstime int64 vswtch, ivswtch; // 仅记录上下文切换次数 // 无 CPU 利用率、 吞吐量、周转时间、等待时间、响应时间统计

Q04_031 · short_answer

若存在 IPI: 发送与处理路径分别在哪些函数/文件? (给关键入口与证据)

项目	方法A	方法B
答案	发送路径: - kernel/main.c:78-79: sbi_send_ipi(mask, 0) - BSP 唤醒 AP - sbi/psicasbi/src/trap/sbi/ipi.rs:28-32: clint::send_ipi(i) - SBI 层 实现 处理路径: - kernel/trap/trap.c:373: sbi_clear_ipi() - 清除 IPI 标志 - sbi/psicasbi/src/main.rs:152: hal::clint::clear_ipi(hartid) - SBI 层 清除	发送路径: sbi_send_ipi() in include/sbi.h 处理路径: 通过 CLINT 软件中断, 在 kernel/trap/trap.c 的 handle_intr() 中处理 INTR_SOFTWARE
证据	[1] kernel/main.c · main for (int i = 1; i < NCPU; i++) { struct sbiret res = sbi_send_ipi(mask, 0); sbi_send_ipi(mask, 0); } [2] sbi/psicasbi/src/trap/sbi/ipi.rs · handler clint::send_ipi(i);	[1] include/sbi.h · sbi_send_ipi static inline struct sbiret sbi_send_ipi(...) { return SBI_CALL_2(IPI_EID, IPI_SEND_IPI, hart_mask, hart_mask_base); } [2] kernel/trap/trap.c · handle_intr else if (INTR_SOFTWARE == scause) { sbi_clear_ipi(); return 0; }

Q04_033 · short_answer

per-CPU 的实现方式是什么? (例如 TLS/tp 寄存器/gsbases/数组索引| hartid; 需证据)

项目	方法A	方法B
答案	数组索引 + tp 寄存器 (include/sched/proc.h:165-168): - cpuid() 通过 r_tp() 读取 tp 寄存器获取当前 hartid - mycpu() 返回 &cpus[cpuid()] - 每核栈空间: kernel/main.c:93-94 中通过 boot_stack + hartid * 4 * PGSIZE 计算	使用 tp 寄存器存储 hartid, 通过 cpuid() 返回 r_tp() 值索引 cpus[] 数组

项目	方法A	方法B
证据	<pre>[1] include/sched/proc.h · cpuid static inline int cpuid(void) { return r_tp(); } [2] kernel/sched/proc.c · mycpu struct cpu *mycpu(void) { int id = cpuid(); return &cpus[id]; }</pre>	<pre>[1] include/sched/proc.h · cpuid static inline int cpuid(void) { return r_tp(); } [2] include/sched/proc.h · mycpu struct cpu *mycpu(void) { int id = cpuid(); return &cpus[id]; } [3] include/hal/riscv.h · r_tp static inline uint64 r_tp() { uint64 x; asm volatile("mv %0, tp" : "=r" (x)); return x; }</pre>

Q04_036 · short_answer

与 03/04/05/08 章的交叉一致性 (Cross-Chapter Consistency), 按以下四项分别作答 (每项须给证据路径或写「单核不适用」) : - 03 TLB: 多核页表修改后 TLB 刷新策略= - 04 调度: 每核运行队列/负载均衡/IPI resched= - 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序= - 08 锁: SpinLock 关中断行为在多核下是否安全=

项目	方法A	方法B
答案	03 TLB: 多核页表修改后 TLB 刷新策略=未发现跨核 TLB 刷新机制, 仅单核 sfence_vma() (kernel/sched/proc.c:675) 04 调度: 每核运行队列/负载均衡/IPI resched=全局 proc_runnable[] 数组 + proc_lock 保护, 无每核队列/负载均衡/IPI resched 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=hart0 先 trapinithart(), AP 上线后各自 trapinithart() (kernel/main.c:53,68) 08 锁: SpinLock 关中断行为在多核下是否安全=是, acquire() 调用 push_off() 关中断 (kernel/sync/spinlock.c:27)	03 TLB: 多核页表修改后 TLB 刷新策略=单核 sfence.vma, 多核无 shutdown 04 调度: 每核运行队列/负载均衡/IPI resched=全局队列, 无负载均衡, 无 IPI resched 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=每核独立 trapinithart(), BSP 先初始化后唤醒 AP 08 锁: SpinLock 关中断行为在多核下是否安全=安全, acquire() 调用 push_off() 关中断
证据	<pre>[1] kernel/sched/proc.c · scheduler w_satp(MAKE_SATP(tmp->pagetable)); sfence_vma(); [2] kernel/sync/spinlock.c · acquire push_off(); // disable interrupts to avoid deadlock.</pre>	<pre>[1] kernel/mm/vm.c · sfence_vma static inline void sfence_vma() { #ifdef QEMU asm volatile("sfence.vma"); #else asm volatile(".word 0x10400073"); asm volatile("fence.i"); #endif } [2] kernel/main.c · main if (hartid == 0) { ... trapinithart(); ... for (int i = 1; i < NCPU; i ++) { sbi_send_ipi(mask, 0); } started = 1; } else { while (started == 0) ; trapinithart(); } [3] kernel/sync/spinlock.c · acquire void acquire(struct spinlock *lk) { push_off(); ...</pre>

Q04_038 · short_answer

NCPU/MAXCPU (或等价宏) 与链接脚本中的每 hart 栈/入口布局是否对应? (搜索 _max_hart_id 等; 给宏定义与链接脚本对应证据, 或写未发现)

项目	方法A	方法B
答案	NCPU=2 (include/param.h:5) 链接脚本: linker/qemu.ld 未显式定义每 hart 栈布局 启动代码: kernel/main.c:93-94 中通过 boot_stack + hartid * 4 * PGSIZE 计算每核栈, 每核 4 页 (16KB) 对应关系: NCPU=2 与 main.c 中 for 循环 (i=1; i<NCPU) 一致, 但链接脚本未显式定义 hart 栈区	NCPU=2 in include/param.h, 链接脚本 linker/linker64.ld 中 boot_stack 分配 409642 字节支持 2 核
证据	[1] include/param.h · NCPU #define NCPU 2 // maximum number of CPUs [2] kernel/main.c · main uint64 kstack = (uint64)boot_stack + hartid * 4 * PGSIZE; freepage_n(kstack, 3);	[1] include/param.h · NCPU #define NCPU 2 [2] kernel/entry.S · boot_stack .section .stack .align 12 .globl boot_stack boot_stack: .space 4096 * 4 * 2 .globl boot_stack_top boot_stack_top: [3] kernel/main.c · main extern char boot_stack[]; uint64 kstack = (uint64)boot_stack + hartid * 4 * PGSIZE; freepage_n(kstack, 3);

05_fs_drivers — 文件系统与设备 I/O

Q05_001 · short_answer

VFS 抽象层 (Virtual File System, VFS)接口是什么形态? (Rust trait / C op 表; 必须给接口定义证据)

项目	方法A	方法B
答案	C 语言函数指针操作表 (op 表) 形态。证据: include/fs/fs.h:43-68 定义了 struct fs_op (超级块操作)、struct inode_op (索引节点操作)、struct dentry_op (目录项操作)、struct file_op (文件操作) 四个操作表结构体, 每个包含函数指针成员如 alloc_inode、create、lookup、read、write 等。具体文件系统在 kernel/fs/fat32/fat32.c:22-40 中实现这些操作表 (如 fat32_inode_op、fat32_file_op)。	C 操作表 (C operation table)。VFS 使用 C 语言的结构体包含函数指针来定义接口, 包括 struct fs_op、struct inode_op、struct dentry_op、struct file_op。
证据	[1] include/fs/fs.h · fs_op/inode_op/dentry_op/file_op 行 43-68 定义四个操作表结构体, 包含函数指针成员如 alloc_inode、create、lookup、read、write 等 [2] kernel/fs/fat32/fat32.c · fat32_inode_op/fat32_file_op 行 22-40 实现 FAT32 文件系统的 inode_op 和 file_op 操作表	[1] include/fs/fs.h · fs_op, inode_op, dentry_op, file_op struct fs_op { struct inode (alloc_inode)(struct superblock sb); void (destroy_inode)(struct inode ip); int (write)(...); int (read)(...); ... }; struct inode_op { struct inode (create)(...); struct inode (lookup)(...); int (truncate)(...); ... }; struct file_op { int (read)(...); int (write)(...); int (*readdir)(...); ... };

Q05_003 · short_answer

若支持 FAT32/Ext4: 它是自研还是第三方库/crate? (必须引用 Cargo.toml/Cargo.lock 或 Makefile 引入证据)

项目	方法A	方法B
答案	自研实现。证据：kernel/fs/fat32/ 目录下包含完整的 FAT32 实现文件（fat32.c、fat.c、dirent.c、cluster.c、fat32.h），无第三方库依赖。项目为纯 C 语言内核（非 Rust），Cargo.toml 仅用于 SBI 固件（sbi/psicasbi/Cargo.toml），内核构建使用 Makefile（行 1-294）直接编译 kernel/fs/fat32/*.c 文件。	自研实现。FAT32 文件系统代码位于 kernel/fs/fat32/目录下，是内核源码的一部分，没有通过 Cargo.toml 或 Makefile 引入第三方库。
证据	[1] kernel/fs/fat32 · fat32_implementation 包含 fat32.c(572 行)、fat.c(394 行)、dirent.c(490 行)、cluster.c(314 行) 等自研 FAT32 实现文件 [2] Makefile · kernel_build 行 1-294 定义 C 语言内核构建规则，直接编译 kernel/fs/fat32/*.c 文件，无第三方 FS 库引入	[1] Makefile · SRC SRC += \$K/fs/fat32/cluster.c \$K/fs/fat32/dirent.c \$K/fs/fat32/fat.c \$K/fs/fat32/fat32.c [2] Cargo.toml · workspace [workspace] members = ["sbi/psicasbi",] - No filesystem crates listed

Q05_004 · short_answer

文件打开路径：文件打开入口（sys_open 或等价）→ VFS 层 → 具体 FS open。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	文件打开调用链：1. sys_openat (kernel/syscall/sysfile.c:253) → 2. nameifrom / create (kernel/fs/fs.c:473 / kernel/fs/fs.c:24) → 3. lookup_path (kernel/fs/fs.c:412) → 4. dirlookup (kernel/fs/fs.c:319) → 5. fat_lookup_dir (FAT32 具体实现，通过 dir->op->lookup 调用) → 6. filealloc (kernel/fs/file.c:32) 分配文件描述符。证据：lsp_get_call_graph 显示 sys_openat 调用 nameifrom → lookup_path → dirlookup，最终通过 inode->op->lookup 调用具体文件系统实现。	1. sys_openat (kernel/syscall/sysfile.c) → 2. nameifrom (kernel/fs/fs.c) → 3. lookup_path (kernel/fs/fs.c) → 4. dirlookup (kernel/fs/fs.c) → 5. fat_lookup_dir (kernel/fs/fat32/dirent.c) → 6. filealloc/fdalloc (kernel/fs/file.c)
证据	[1] kernel/syscall/sysfile.c · sys_openat 行 253 开始，解析参数后调用 nameifrom 或 create 进行路径解析或文件创建 [2] kernel/fs/fs.c · lookup_path/dirlookup 行 412 实现路径解析，行 319 实现目录查找，支持 . 和 .. 特殊目录项 [3] kernel/fs/file.c · filealloc 行 32 分配 struct file 结构体，初始化引用计数和锁	[1] kernel/syscall/sysfile.c · sys_openat if ((ip = nameifrom(dp, path)) == NULL) { return -ENOENT; } ... f->ip = ip; f->type = FD_INODE; [2] kernel/fs/fs.c · nameifrom, lookup_path, dirlookup struct inode nameifrom(struct inode ip, char path) { return lookup_path(ip, path, 0, name); } ... struct inode dirlookup(struct inode dir, char filename, uint poff) { ... ip = dir->op->lookup(dir, filename, poff); } [3] kernel/fs/fat32/dirent.c · fat_lookup_dir struct inode fat_lookup_dir(struct inode dir, char filename, uint poff) { ... struct fat32_entry ep = fat_lookup_dir_ent(dir, filename, &off); ... ip = &ep->vfs_inode; }

Q05_005 · short_answer

文件描述符表 (File Descriptor Table, FD Table) 的实现形态是什么？（固定数组/Vec/BTreeMap 等；必须给结构体定义证据）

项目	方法A	方法B
答案	固定数组 + 链表扩展形态。证据：include/fs/file.h:29-37 定义 struct fdtable 包含 struct file *arr[NOFILE] 固定大小数组（NOFILE 为常量），并通过 struct fdtable *next 指针支持多表链表扩展。 kernel/fs/file.c:434-470 的 fdalloc 函数实现：当当前表满时（nextfd == NOFILE），分配新表并链接到链表。	固定数组。struct fdtable 使用固定大小的数组 struct file *arr[NOFILE]，其中 NOFILE=16 定义在 param.h 中。
证据	[1] include/fs/file.h · fdtable 行 29-37 定义 fdtable 结构体，包含 arr[NOFILE] 固定数组和 next 指针支持链表扩展 [2] kernel/fs/file.c · fdalloc 行 434-470 实现文件描述符分配，表满时通过 newfdtable 分配新表并链接	[1] include/fs/file.h · fdtable struct fdtable { uint16 basefd; uint16 nextfd; uint16 used; uint16 exec_close; struct file arr[NOFILE]; struct fdtable next; }; [2] include/param.h · NOFILE #define NOFILE 16 // open files per process

Q05_007 · short_answer

若存在缓存：驱逐策略是什么（LRU/Clock/FIFO/无驱逐）？必须指出判断依据（字段/算法分支）证据。

项目	方法A	方法B
答案	LRU（最近最少使用）策略。证据：kernel/fs/bio.c:84-147 的 bget 函数实现：1.使用 d_list lru_head（行 69）作为 LRU 链表，未使用缓冲在链表尾部；2. 缓存命中时将缓冲移到链表头部（行 105-109：_list_remove + _list_push_front）；3. 驱逐时从链表尾部获取（行 122-132：lru_head.prev），符合 LRU 特征。	LRU (Least Recently Used)。通过 lru_head 双向链表实现，最近使用的 buffer 移到链表头部，驱逐时从尾部获取。bget() 中从 lru_head.prev 获取最久未使用的 buffer。
证据	[1] kernel/fs/bio.c · bget 行 105-109 缓存命中时移到头部，行 122-132 从尾部驱逐，实现 LRU 策略 [2] kernel/fs/bio.c · lru_head 行 69 定义 LRU 链表头，用于追踪缓冲使用顺序	[1] kernel/fs/bio.c · binit, bget, bput static struct d_list lru_head; ... for (struct buf b = bufs; b < bufs + BNUM; b++) { dlist_add_before(&lru_head, &b->list); } ... struct d_list dl = lru_head.prev; b = container_of(dl, struct buf, list); ... bput(): dlist_add_after(&lru_head, &b->list);

Q05_010 · tri_state_impl

是否实现 poll/select/epoll（或等价事件机制）？（必须三态）

项目	方法A	方法B
答案	stub	implemented

项目	方法A	方法B
证据	[1] kernel/fs/poll.c · poll_implementation 存在 poll.c(243 行) 实现 poll 等待队列管理, 但 grep 搜索 sys_poll 未找到系统调用入口, 仅有内部 poll 框架无用户态接口 [2] kernel/syscall · sys_poll_scan 在 syscall 目录搜索 sys_poll/sys_select/sys_epoll 均未找到, poll 功能仅有内部实现无系统调用暴露	[1] kernel/fs/poll.c · ppoll, pselect int ppoll(struct pollfd pfd, int nfd, struct timespec timeout, __sigset_t sigmask) { ... } int pselect(int nfd, struct fdset readfd, struct fdset writefd, struct fdset exceptfd, struct timespec timeout, __sigset_t sigmask) { ... } [2] kernel/syscall/sysfile.c · sys_pselect, sys_ppoll uint64 sys_pselect(void) { ... int ret = pselect(nfd, ...); } uint64 sys_ppoll(void) { ... int ret = ppoll(pfd, nfd, ...); }

Q05_012 · tri_state_impl

是否支持符号链接 (symlink) 的解析/跟随? (必须三态)

项目	方法A	方法B
答案	stub	not_found
证据	[1] kernel/syscall/sysfile.c · sys_readlinkat 行 698-735 实现 sys_readlinkat, 但仅返回路径字符串 (行 725-728 调用 namepath), 未在 open 路径中实现 symlink 跟随逻辑 [2] kernel/fs/fs.c · symlink_follow_scan 在 lookup_path 和 dirlookup 中未找到 S_IFLNK 符号链接类型检查或跟随逻辑	[1] include/fs/stat.h · S_IFLNK, S_ISLNK #define S_IFLNK 0120000 // symbolic link #define S_ISLNK(m) (((m) & S_IFMT) == S_IFLNK) [2] kernel/fs/fs.c · symlink, readlink 在附件源码中检索 symlink/readlink 相关实现, 仅在 stat.h 中定义了 S_IFLNK 宏, 但未发现符号链接的创建、解析或跟随的实际实现代码。

Q05_017 · single_choice

文件数据块分配方式 (File Allocation Method, Stallings Ch12) 更接近哪种?

项目	方法A	方法B
答案	FAT 表内嵌空闲链 (FAT32 特有)	B. 链式分配 (Chained/Linked Allocation): 块通过指针链接
证据	[1] kernel/fs/fat32/fat.c · fat_sec_of_clus/fat_offset_of_clus 行 20-30 实现簇号到 FAT 表扇区偏移的转换, FAT32 使用 FAT 表链式管理簇分配 [2] kernel/fs/fat32/fat32.h · fat32_sb 行 40-60 定义 FAT32 超级块包含 bpb.fat_sz (FAT 表大小)、bpb.root_clus (根簇号) 等 FAT 特有字段	[1] kernel/fs/fat32/fat.c · read_fat, write_fat uint32 read_fat(struct superblock sb, uint32 cluster) { ... next = (uint32 *) (fat->fatcache.page + idx * SECSZ + sec_off); ... } FAT32 uses FAT chain where each entry points to next cluster. [2] kernel/fs/fat32/cluster.c · reloc_clus, fill_clus_table clus = read_fat(sb, preclus); // get next cluster number ... table->clus[i] = clus; // FAT chain traversal

Q05_023 · short_answer

驱动框架接口是什么? (Rust Driver trait / C driver ops / 注册表; 必须引用接口定义证据)

项目	方法A	方法B
答案	C 语言直接初始化模式，无统一驱动框架接口。证据： kernel/hal/ 目录下各驱动（ virtio_disk.c、 sdcard.c、 plic.c）均通过独立的 *_init 函数直接初始化（如 virtio_disk_init、 sdcard_init、 plicinit），无 driver_ops 结构体或注册表机制。 kernel/main.c:60-66 直接顺序调用 disk_init()、 binit() 等初始化函数。	C 函数接口。驱动通过直接调用 C 函数实现，没有统一的 driver ops 表或注册机制。每个驱动模块（uart, disk, plic 等）有独立的初始化函数。
证据	[1] kernel/hal/virtio_disk.c · virtio_disk_init 行 95-165 直接初始化 VirtIO 磁盘，无注册回调 [2] kernel/main.c · main 行 60-66 直接调用 disk_init()、 binit() 等，无驱动注册/探测机制	[1] kernel/hal/disk.c · disk_init, disk_read, disk_write void disk_init(void) { #ifdef QEMU virtio_disk_init(); #else sdcard_init(); #endif } int disk_read(struct buf b) { ... } [2] kernel/hal/virtio.h · virtio_disk_init, virtio_disk_read void virtio_disk_init(void); int virtio_disk_read(struct buf b); void virtio_disk_write_start(void); // Direct function calls, no ops table

Q05_024 · short_answer

驱动注册与初始化顺序是什么？（init_drivers/probe/driver_manager 等；列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	无驱动注册机制，采用硬编码顺序初始化。关键节点： 1. plicinit（ kernel/hal/plic.c:20）→ 2. plicinithart（ kernel/hal/plic.c:33）→ 3. disk_init（ kernel/hal/disk.c）→ 4. binit（ kernel/fs/bio.c:84）→ 5. rootfs_init（ kernel/fs/rootfs.c:280）。 证据： kernel/main.c:58-66 在 hart 0 上顺序调用 plicinit() → disk_init() → binit() → rootfs_init()（通过 do_mount 挂载 FAT32）。	1. cpuinit() → 2. consoleinit() → 3. kpmininit() → 4. kvmininit() → 5. trapinithart() → 6. plicinit() → 7. disk_init() → 8. binit()。在 kernel/main.c 的 main() 函数中按固定顺序初始化。
证据	[1] kernel/main.c · main 行 58-66 在 hart 0 初始化流程： plicinit → disk_init → binit → rootfs_init [2] kernel/hal/plic.c · plicinit/plicinithart 行 20-31 初始化 PLIC 中断使能，行 33-60 按 hart 启用中断	[1] kernel/main.c · main if (hartid == 0) { cpuinit(); consoleinit(); printfinit(); kpmininit(); kvmininit(); kmalloccinit(); trapinithart(); procinit(); plicinit(); #ifndef QEMU fpioa_pin_init(); dmac_init(); #endif disk_init(); binit(); ... }

Q05_029 · short_answer

MMIO 地址来源是什么？（DTB 提供 / 常量硬编码 / 物理→虚拟转换；必须给证据）

项目	方法A	方法B
答案	常量硬编码 + 物理→虚拟转换。证据：include/memlayout.h:40-68 硬编码物理地址（如 UART 0x10000000/0x38000000、VIRTIO0 0x10001000、PLIC 0x0c000000），并通过 *_V 宏（如 UART_V = UART + VIRT_OFFSET，VIRT_OFFSET=0x3F0000000L）转换为虚拟地址。kernel/mm/vm.c:59 的 kvmmap(UART_V, UART, PGSIZE, ...) 建立物理到虚拟的页表映射。	常量硬编码。MMIO 地址在 include/memlayout.h 中定义为宏常量，根据平台 (QEMU/k210) 使用条件编译选择不同值。
证据	[1] include/memlayout.h · UART/VIRTIO0/PLIC 行 40-68 硬编码物理地址并定义 *_V 虚拟地址宏（物理地址 + VIRT_OFFSET） [2] kernel/mm/vm.c · kvminit 行 59 调用 kvmmap(UART_V, UART, PGSIZE, ...) 建立 MMIO 区域的页表映射	[1] include/memlayout.h · UART, CLINT, PLIC, VIRTIO0 #ifdef QEMU #define UART 0x10000000L #else #define UART 0x38000000L #endif #define UART_V (UART + VIRT_OFFSET) #define CLINT 0x02000000L #define PLIC 0x0c000000L #ifdef QEMU #define VIRTIO0 0x10001000 #endif

Q05_030 · short_answer

多平台适配是如何通过构建/条件编译选择驱动的？（features/Kconfig/Makefile 规则；必须给证据）

项目	方法A	方法B
答案	Makefile 平台宏 + #ifdef QEMU 条件编译。证据：Makefile:1-2 通过 platform := k210 或 platform := qemu 选择平台，行 34-36 添加 -D QEMU 编译标志。include/memlayout.h:40-45 使用 #ifdef QEMU 区分 UART 地址（QEMU: 0x10000000, K210: 0x38000000）。kernel/hal/plic.c:42-58 使用 #ifdef QEMU 区分 S 模式/M 模式中断使能。	Makefile 条件编译。通过 platform 变量 (k210/qemu) 控制，使用 #ifdef QEMU / #ifndef QEMU 条件编译选择不同驱动代码。
证据	[1] Makefile · platform_select 行 1-2 定义 platform 变量，行 34-36 根据 platform 添加-D QEMU 编译标志 [2] include/memlayout.h · UART_V 行 40-45 使用#ifdef QEMU 区分 UART 物理地址	[1] Makefile · platform, CFLAGS platform := k210 #platform := qemu ... ifeq \$(platform), qemu CFLAGS += -D QEMU endif ... ifeq \$(platform), k210) SBI := ./sbi/sbi-k210 SRC += \$K/hal/spi.c \$K/hal/sdcard.c ... else SRC += \$K/hal/virtio_disk.c endif [2] kernel/hal/disk.c · disk_init void disk_init(void) { #ifdef QEMU virtio_disk_init(); #else sdcard_init(); #endif }

Q05_031 · tri_state_impl

是否存在 MMU 启用前后串口地址切换（phys/virt 切换）逻辑？（必须三态）

项目	方法A	方法B
答案	implemented	not_found

项目	方法A	方法B
证据	[1] include/sbi.h · sbi_console_putchar 行 24-26 通过 SBI 调用实现早期输出（MMU 启用前使用物理地址） [2] kernel/console.c · consolewrite 行 50-80 MMU 启用后使用 UART_V 虚拟地址，通过 consputc → sbi_console_putchar 输出 [3] kernel/mm/vm.c · kvminit 行 59 调用 kvmmap(UART_V, UART, PGSIZE, ...) 在 MMU 启用前建立 UART 物理到虚拟的映射	[1] kernel/console.c · UART_V, consputc consputc() 使用 sbi_console_putchar() 通过 SBI 调用输出，不直接访问 UART MMIO。未发现 MMU 启用前后的 UART 地址切换逻辑。 [2] include/memlayout.h · UART_V, VIRT_OFFSET #define UART_V (UART + VIRT_OFFSET) #define VIRT_OFFSET 0x3F00000000L - 定义了虚拟地址映射，但未发现运行时切换逻辑

Q05_034 · single_choice

I/O 控制技术 (I/O Control Techniques, Stallings Ch11) 更接近哪种？

项目	方法A	方法B
答案	中断驱动 I/O (Interrupt-Driven I/O): 设备完成后发中断通知 CPU	D. 混合 (小传输用中断, 大传输用 DMA)
证据	[1] kernel/hal/virtio_disk.c · virtio_disk_intr VirtIO 驱动通过中断通知完成（行 252-280），检查 used->idx 更新并唤醒等待队列 [2] kernel/trap/trap.c · handle_intr 行 287-402 处理外部中断，DISK_IRQ 触发时调用 virtio_disk_intr	[1] kernel/hal/virtio_disk.c · virtio_disk_intr Interrupt-driven completion notification. virtio_disk_intr() is called on disk completion interrupt, wakes up waiting processes. [2] kernel/hal/dmac.c · dmac_set_single_mode, dmac_intr DMA controller for memory transfers. dmac_set_single_mode() configures DMA channels. dmac_intr() handles DMA completion interrupts. Used by sdcard and spi drivers for data transfers. [3] kernel/hal/sdcard.c · sdcard_read, sdcard_write sd_read_data_dma() and sd_write_data_dma_no_wait() use DMA for block transfers. Interrupt handler sdcard_intr() processes completion.

06_sync_ipc — 同步互斥与进程间通信

Q06_001 · short_answer

该内核提供了哪些同步原语？（SpinLock/Mutex/RwLock/Semaphore/Condvar/WaitQueue 等；列出类型定义证据）

项目	方法A	方法B
答案	已实现 SpinLock（自旋锁）、SleepLock（睡眠锁）、WaitQueue（等待队列）。证据： include/sync/spinlock.h:7-13 定义 struct spinlock（含 locked、name、cpu 字段）； include/sync/sleeplock.h:9-16 定义 struct sleeplock（含 locked、lk、name、pid 字段）； include/sync/waitqueue.h:16-24 定义 struct wait_queue（含 lock、head）和 struct wait_node（含 chan、list）。未发现 RwLock、Semaphore、Condvar 实现。	SpinLock: include/sync/spinlock.h struct spinlock { uint locked; char name; struct cpu cpu; }; SleepLock: include/sync/sleeplock.h struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; }; WaitQueue: include/sync/waitqueue.h struct wait_queue { struct spinlock lock; struct d_list head; }; 未发现 RwLock/Semaphore/Condvar 实现

项目	方法A	方法B
证据	[1] include/sync/spinlock.h · spinlock struct spinlock { uint locked; char name; struct cpu cpu; }; [2] include/sync/sleeplock.h · sleeplock struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; }; [3] include/sync/waitqueue.h · wait_queue struct wait_queue { struct spinlock lock; struct d_list head; };	[1] include/sync/spinlock.h · spinlock struct spinlock { uint locked; char name; struct cpu cpu; }; [2] include/sync/sleeplock.h · sleeplock struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; }; [3] include/sync/waitqueue.h · wait_queue struct wait_queue { struct spinlock lock; struct d_list head; };

Q06_004 · fill_in

sleep / wakeup 不变量 (Sleep-Wakeup Invariant) 分析, 按格式填写: - sleep 入口函数: **(路径)** - **入睡前持有的锁:** (无则写 none) - 防丢 wakeup (Lost Wakeup Prevention) 机制: **(如: 持队列锁检查条件 / 无防护)** - **wakeup 函数:** (路径) - 唤醒与锁释放顺序: __ (先唤醒后释放 / 先释放后唤醒 / 其他)

项目	方法A	方法B
答案	sleep 入口函数: kernel/sched/proc.c:569 (sleep) 入睡前持有的锁: 必须持有 proc_lock 或传入的 lk (若 lk 非 proc_lock 则释放 lk 后持 proc_lock 进入 sleep) 防丢 wakeup (Lost Wakeup Prevention) 机制: 持 proc_lock 检查条件并进入睡眠, wakeup 也持 proc_lock 遍历睡眠队列, 确保原子性 wakeup 函数: kernel/sched/proc.c:386 (wakeup) 唤醒与锁释放顺序: 先唤醒 (__wakeup_no_lock 将进程移入 runnable 队列) 后释放 proc_lock	sleep 入口函数: kernel/sched/proc.c::sleep() 入睡前持有的锁: 传入的 lk (调用者持有) + proc_lock (内部获取) 防丢 wakeup (Lost Wakeup Prevention) 机制: 持队列锁检查条件 / 在 proc_lock 保护下插入睡眠队列 wakeup 函数: kernel/sched/proc.c::wakeup() 唤醒与锁释放顺序: 先唤醒后释放 (在 proc_lock 保护下遍历睡眠队列并唤醒)
证据	[1] kernel/sched/proc.c · sleep void sleep(void chan, struct spinlock lk) { ... if (&proc_lock != lk) { __enter_proc_cs release(lk); } ... sched(); __leave_proc_cs acquire(lk); } [2] kernel/sched/proc.c · wakeup void wakeup(void *chan) { __enter_proc_cs __wakeup_no_lock(chan); __leave_proc_cs }	[1] kernel/sched/proc.c · sleep void sleep(void chan, struct spinlock lk) { if (&proc_lock != lk) { __enter_proc_cs release(lk); } p->chan = chan; __remove(p); __insert_sleep(p); sched(); p->chan = NULL; __leave_proc_cs acquire(lk); } [2] kernel/sched/proc.c · wakeup void wakeup(void chan) { __enter_proc_cs __wakeup_no_lock(chan); __leave_proc_cs } static void __wakeup_no_lock(void chan) { struct proc p = proc_sleep; while (NULL != p) { struct proc next = p-> sched_next; if ((uint64)chan == (uint64)p->chan) { __remove(p); p-> timer = TIMER_IRQ; p->chan = NULL; __insert_runnable(PRIORITY_IRQ, p); } p = next; } }

Q06_011 · short_answer

若实现 signal handler: 用户态 handler 上下文如何构建? 是否存在 sigreturn 恢复原 trap frame? (必须给证据)

项目	方法A	方法B
答案	用户态 handler 上下文构建: 在 kernel/sched/signal.c:177-264 的 sighandle() 中, 分配 sig_frame 保存原 trapframe (frame->tf = p->trapframe), 新建 trapframe 设置 epc 为 sig_trampoline 中的 sig_handler 地址 (tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline))), a0 传递 signum, a1 传递 handler 地址. sigreturn 恢复: kernel/sched/signal.c:267-280 的 sigreturn() 从 sig_frame 链表中取出原 trapframe 恢复 (p->trapframe = frame->tf), 释放 sig_frame。证据: kernel/sched/signal.c:237-254 构建上下文, kernel/sched/signal.c:267-280 恢复, kernel/trap/sig_trampoline.S:8-13 提供 trampoline 代码。	用户态 handler 上下文构建: 在 kernel/sched/signal.c::sighandle() 中, 分配 sig_frame 结构保存当前 trapframe, 设置新的 trapframe 指向 sig_trampoline 中的 sig_handler. sigreturn 恢复: 存在, 在 kernel/sched/signal.c::sigreturn() 中从 sig_frame 链表中恢复之前的 trapframe 和信号掩码
证据	[1] kernel/sched/signal.c · sighandle frame->tf = p->trapframe; tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline)); tf->a0 = signum; tf->a1 = (uint64)(sigact->sigact->__sigaction_handler.sa_handler); [2] kernel/sched/signal.c · sigreturn struct sig_frame *frame = p->sig_frame; ... p->trapframe = frame->tf; p->sig_frame = frame->next; kfree(frame); [3] kernel/trap/sig_trampoline.S · sig_handler sig_handler: jalr a1; li a7, SYS_rt_sigreturn; ecall	[1] kernel/sched/signal.c · sighandle frame = kmalloc(sizeof(struct sig_frame)); tf = kmalloc(sizeof(struct trapframe)); for (int i = 0; i < SIGSET_LEN; i++) { frame->mask.__val[i] = p->sig_set.__val[i]; ... } frame->tf = p->trapframe; tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline)); tf->sp = p->trapframe->sp; tf->a0 = signum; ... p->trapframe = tf; frame->next = p->sig_frame; p->sig_frame = frame; [2] kernel/sched/signal.c · sigreturn void sigreturn(void) { struct proc p = myproc(); if (NULL == p->sig_frame) { exit(-1); } struct sig_frame frame = p->sig_frame; for (int i = 0; i < SIGSET_LEN; i++) { p->sig_set.__val[i] = frame->mask.__val[i]; } kfree(p->trapframe); p->trapframe = frame->tf; p->sig_frame = frame->next; kfree(frame); }

Q06_013 · single_choice

底层原子操作来源更接近哪种？

项目	方法A	方法B
答案	自定义汇编 (ldxr/stxr、lock xchg 等)	A. Rust core::sync::atomic (标准库)
证据	[1] kernel/sync/spinlock.c · acquire while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // 注释说明: On RISC-V, sync_lock_test_and_set turns into an atomic swap: amoswap.w.aq [2] kernel/sync/spinlock.c · release __sync_lock_release(&lk->locked); // On RISC-V, sync_lock_release turns into an atomic swap: amoswap.w zero, zero, (s1)	[1] kernel/sync/spinlock.c · acquire while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; ... __sync_synchronize(); ... __sync_lock_release(&lk->locked); [2] kernel/sync/spinlock.c · atomic_ops On RISC-V, sync_lock_test_and_set turns into an atomic swap ... This code doesn't use a C assignment, since the C standard implies that an assignment might be implemented with multiple store instructions.

Q06_014 · short_answer

死锁四必要条件 (Coffman Conditions) 在该内核中是否均成立？请逐条作答（互斥 Mutual Exclusion / 持有并等待 Hold-and-Wait / 不可剥夺 No Preemption / 循环等待 Circular Wait），并结合 SpinLock/Mutex 的实现给出证据或写「不适用」。

项目	方法A	方法B
答案	1. 互斥 (Mutual Exclusion): 成立。SpinLock 通过原子操作 <code>__sync_lock_test_and_set</code> 确保同一时刻仅一个 CPU 持有锁 (<code>kernel/sync/spinlock.c:34</code>)。SleepLock 在 spinlock 基础上增加睡眠机制，同样保证互斥 (<code>kernel/sync/sleeplock.c:22-28</code>)。 2. 持有并等待 (Hold-and-Wait): 成立。sleep() 实现中，进程持有 <code>proc_lock</code> 进入睡眠 (<code>kernel/sched/proc.c:577-580</code>)，pipelock 中进程持有队列锁调用 sleep (<code>kernel/fs/pipe.c:90-97</code>)。 3. 不可剥夺 (No Preemption): 成立。锁只能由持有者主动释放 (<code>release/releasesleep</code>)，内核无强制剥夺锁的机制。 4. 循环等待 (Circular Wait): 可能成立。内核未实现全局锁顺序规范 (见 Q06_016)，存在嵌套锁场景 (如 pipe 操作中同时持有 <code>pi->lock</code> 和 <code>wq->lock</code>)，理论上可能形成 ABBA 死锁。	互斥 Mutual Exclusion: 成立。spinlock 使用 <code>__sync_lock_test_and_set</code> 保证同一时刻只有一个 CPU 持有锁。持有并等待 Hold-and-Wait: 成立。sleep() 函数中进程持有 lk 锁时调用 sched() 放弃 CPU，可能等待其他资源。不可剥夺 No Preemption: 成立。锁只能由持有者主动 release() 释放，内核不会强制剥夺。循环等待 Circular Wait: 可能成立。代码中未发现全局锁顺序规范，存在嵌套锁导致循环等待的可能性 (如 pipe 操作中同时持有 <code>pi->lock</code> 和 <code>queue->lock</code>)。
证据	[1] <code>kernel/sync/spinlock.c · acquire</code> <code>while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // 原子操作保证互斥</code> [2] <code>kernel/sched/proc.c · sleep</code> <code>if (&proc_lock != lk) { __enter_proc_cs</code> <code>release(lk); } ... sched(); // 持锁进入睡眠</code> [3] <code>kernel/fs/pipe.c · pipelock</code> <code>acquire(&q->lock); wait_queue_add(q, wait);</code> <code>while (!wait_queue_is_first(q, wait)) {</code> <code>sleep(wait->chan, &q->lock); }</code>	[1] <code>kernel/sync/spinlock.c · acquire</code> <code>while(__sync_lock_test_and_set(&lk->locked, 1) != 0) ; ...</code> <code>lk->cpu = mycpu();</code> [2] <code>kernel/sched/proc.c · sleep</code> <code>void sleep(void chan, struct spinlock lk) { ...</code> <code>acquire(&proc_lock); ... release(lk); ... __insert_sleep(p);</code> <code>sched(); ... acquire(lk); }</code> [3] <code>kernel/fs/pipe.c · pipelock</code> <code>static void pipelock(struct pipe pi, struct wait_node wait,</code> <code>int who) { ... acquire(&q->lock); wait_queue_add(q, wait);</code> <code>while (!wait_queue_is_first(q, wait)) { sleep(wait->chan,</code> <code>&q->lock); } release(&q->lock); }</code>

Q06_017 · tri_state_impl

是否实现管程/条件变量 (Monitor / Condition Variable, Stallings Ch5)? (必须三态; 搜索 Condvar / condition_variable / monitor / wait/notify/signal 等; 若 implemented 需区分 Hoare 语义 (等待者立即恢复) vs Mesa 语义 (等待者重新竞争锁))

项目	方法A	方法B
答案	not_found	implemented

项目	方法A	方法B
证据	[1] include/sync/ · grep_condvar 检索 Condvar/condition_variable/monitor/pthread_cond, 仅在 xv6-user/run_test.c 发现测试用例引用, 内核源码中无实现 [2] kernel/sync/ · sync_module_scan kernel/sync/ 仅包含 spinlock.c 和 sleeplock.c, 无条件变量实现	[1] kernel/sched/proc.c · sleep void sleep(void chan, struct spinlock lk) { ... __insert_sleep(p); sched(); ... } // 进程释放锁后进入睡眠, 唤醒后需重新获取锁 [2] kernel/sched/proc.c · wakeup void wakeup(void *chan) { ... __wakeup_no_lock(chan); ... } // 唤醒的进程被插入 runnable 队列, 需重新竞争锁, 符合 Mesa 语义 [3] kernel/fs/pipe.c · pipelock while (!wait_queue_is_first(q, wait)) { sleep(wait->chan, &q->lock); } // 唤醒后需重新检查条件, Mesa 语义

Q06_018 · short_answer

经典同步问题验证 (Classic Synchronization Problems, Stallings Ch5): 以下三个经典问题在该内核中是否有对应实现或测试? - 生产者-消费者 (Producer-Consumer / Bounded Buffer): **(implemented/notfound + 证据) - 读者-写者 (Readers-Writers):** _ (实现了读者优先/写者优先/公平? + 证据) - 哲学家就餐 (Dining Philosophers): __ (implemented/not_found)

项目	方法A	方法B
答案	生产者 - 消费者 (Producer-Consumer / Bounded Buffer): not_found (管道 pipe 实现了有界缓冲区的生产者 - 消费者模式, 但无专门的测试或示例代码; kernel/fs/pipe.c 使用 PIPESIZE=1024 的环形缓冲, pipewrite 为生产者, piperead 为消费者) 读者 - 写者 (Readers-Writers): not_found (未实现读写锁, 无读者 - 写者问题的专门实现或测试; xv6-user/run_test.c 中有 pthread_rwlock 测试但内核无对应实现) 哲学家就餐 (Dining Philosophers): not_found (全仓库检索 dining/philosopher 关键词, 0 命中)	生产者 - 消费者 (Producer-Consumer / Bounded Buffer): implemented - kernel/fs/pipe.c 中的 pipe 实现使用环形缓冲区和 wait_queue 实现生产者 - 消费者模式 读者 - 写者 (Readers-Writers): not_found - 未找到读写锁或读者 - 写者问题测试代码 哲学家就餐 (Dining Philosophers): not_found - 未找到哲学家就餐问题测试代码
证据	[1] kernel/fs/pipe.c · pipe struct pipe { char data[PIPESIZE]; uint nread, nwrite; ... }; // 有界缓冲区, 但未作为经典问题示例 [2] xv6-user/run_test.c · pthread_test run_test.c 中有 pthread_rwlock/pthread_cond 测试用例, 但内核未实现对应原语	[1] kernel/fs/pipe.c · pipewrite int pipewrite(struct pipe pi, uint64 addr, int n) { ... while ((m = pipewritable(pi)) == PIPESIZE) { ... sleep(wait->chan, &pi->lock); } ... pipewakeup(pi, PIPE_READER); ... } [2] kernel/fs/pipe.c · piperead int piperead(struct pipe pi, uint64 addr, int n) { ... while ((m = pipereadable(pi)) == 0 && pi->writeopen) { ... sleep(wait->chan, &pi->lock); } ... pipewakeup(pi, PIPE_WRITER); ... } [3] xv6-user · readers_writers 在 xv6-user 目录下搜索 readers_writers/dining_philosophers 等测试程序, 未找到相关实现

07_security — 安全机制与权限模型

Q07_002 · tri_state_impl

是否存在凭证/权限数据结构（UID/GID/Credential/Capability/ACL 等）？（必须三态）

项目	方法A	方法B
答案	stub	implemented
证据	[1] include/sched/proc.h · struct proc 进程控制块 struct proc 中无 uid/gid 字段，仅包含 pid、state、pagetable、fds 等，无凭证相关字段 [2] include/fs/stat.h · struct kstat struct kstat 包含 uid/gid 字段 (uint32 uid; uint32 gid;)，但仅用于 stat 返回，非进程凭证 [3] kernel/syscall/sysproc.c · sys_getuid uint64 sys_getuid(void) { return 0; } 硬编码返回 0，无真实 UID 支持	[1] include/fs/stat.h · kstat struct kstat { ... uint32 uid; uint32 gid; ... }; [2] include/elf.h · AT_UID/AT_EUID/AT_GID/AT_EGID #define AT_UID 11 #define AT_EUID 12 #define AT_GID 13 #define AT_EGID 14 [3] kernel/syscall/sysproc.c · sys_getuid/sys_geteuid/sys_getgid/sys_getegid uint64 sys_getuid(void) { return 0; } // 所有 UID/GID 相关调用返回 0

Q07_004 · short_answer

若存在权限检查：入口点与核心检查函数链路是什么？（列 2-5 个节点并给证据）

项目	方法A	方法B
答案	权限检查链路（简化版，假设 root）：sys_faccessat → nameifrom → ip->mode >> 6 检查所有者权限。证据： kernel/syscall/sysfile.c:1057-1072 sys_faccessat 中'assume user as root'注释后直接检查 (imode & mode) != mode; kernel/syscall/sysfile.c:466-530 sys_openat 中仅检查 S_ISDIR 与 omode 匹配，无 UID 检查。	未发现完整的权限检查链路。sys_openat -> namei -> lookup_path -> dirlookup 路径中仅有文件存在性检查，无基于 UID/GID 的访问控制检查。faccessat 系统调用存在但仅检查文件模式位，未与进程凭证关联。
证据	[1] kernel/syscall/sysfile.c · sys_faccessat ip = nameifrom(dp, path); ... // assume user as root; int imode = (ip->mode >> 6) & 0x7; if ((imode & mode) != mode) return -1; [2] kernel/syscall/sysfile.c · sys_openat if (S_ISDIR(ip->mode) && (omode & (O_WRONLY O_RDWR))) return -EISDIR; 仅类型检查，无权限验证	[1] kernel/syscall/sysfile.c · sys_faccessat int sys_faccessat(void) { ... if ((imode & mode) != mode) return -1; } // 仅检查模式位，未检查进程 UID/GID [2] kernel/fs/fs.c · namei/lookup_path 路径查找函数中无权限验证逻辑

Q07_009 · short_answer

本项目支持哪些架构（riscv64/aarch64/x86_64/loongarch64 等）？每种架构的安全相关初始化（特权级配置、PMP/MPU/SMEP 等）是否有代码证据？（必须逐架构作答，无证据写「未发现」）

项目	方法A	方法B
答案	仅支持 RISC-V 64 (riscv64) 。证据: include/hal/riscv.h 定义 SSTATUS_SPP/SSTATUS_PUM/SSTATUS_SUM 等 RISC-V 特有寄存器位; sbi/psicasbi/src/main.rs:161-187 有 PMP 初始化代码但配置为允许所有访问 (pmpaddr = -1, pmpcfg = R W X) ; 未发现 aarch64/x86_64/loongarch64 相关代码。	仅支持 riscv64 架构。证据: 1) Makefile 中 TOOLPREFIX := riscv64-linux-gnu- 2) include/hal/riscv.h 中定义 RISC-V 特有 CSR 操作 3) sbi/psicasbi/Cargo.toml 中 target = riscv64imac-unknown-none-elf。安全相关初始化: PMP 配置在 sbi/psicasbi/src/main.rs 中有初始化代码 (pmpcfg0/pmpaddr0 设置) , 但内核中未见 S-mode 特权级保护机制 (如 PMP/SMEP 等价物) 的进一步配置。
证据	[1] include/hal/riscv.h · SSTATUS_SPP/SSTATUS_PUM RISC-V sstatus 寄存器位定义: SSTATUS_SPP (1L << 8), SSTATUS_PUM (1L << 18) [2] sbi/psicasbi/src/main.rs · PMP_initialization PMP 初始化: pmpc = PMP_NAPOT PMP_R PMP_W PMP_X; pmpaddr = -1; 允许所有内存访问 [3] linker/qemu.ld · ENTRY ENTRY(_entry) 针对 RISC-V QEMU 平台; linker/k210.ld 针对 K210 板级	[1] Makefile · TOOLPREFIX TOOLPREFIX := riscv64-linux-gnu- [2] sbi/psicasbi/Cargo.toml · target target = "riscv64imac-unknown-none-elf" [3] sbi/psicasbi/src/main.rs · PMP initialization PMP initialization started... pmpc: PMP_NAPOT PMP_R PMP_W PMP_X... csrwr pmpaddr0, -1size as usize... PMP initialization completed. [4] include/hal/riscv.h · RISC-V CSRs 定义 mstatus/sstatus/satp/stvec 等 RISC-V 特有控制寄存器操作

Q07_012 · short_answer

UID/GID 字段是否在 syscall 路径上真实执行权限检查? (搜索 check_perm/inode_permission; 若只有字段无检查链须标注「仅有定义但未强制执行 ♦」; 给检查链证据或写「字段存在但无检查链」)

项目	方法A	方法B
答案	字段存在但无检查链。struct kstat 包含 uid/gid 字段 (include/fs/stat.h:54-55) , 但 struct proc 无 uid/gid 字段; sys_faccessat 中注释'assume user as root'后直接检查 mode 位 (kernel/syscall/sysfile.c:1065-1070) , 无 UID/GID 比较逻辑; sys_getuid 硬编码返回 0 (kernel/syscall/sysproc.c:253-255) 。	字段存在但无检查链。kstat 结构体中有 uid/gid 字段, sys_getuid 等系统调用返回硬编码的 0, faccessat 仅检查文件模式位 (imode & mode), 未与进程 UID/GID 进行比较。openat/execve/write 等关键系统调用路径中未见基于凭证的权限验证逻辑。
证据	[1] include/fs/stat.h · struct kstat struct kstat { ... uint32 uid; uint32 gid; ... } 仅用于 stat 返回 [2] kernel/syscall/sysfile.c · sys_faccessat // assume user as root; int imode = (ip->mode >> 6) & 0x7; if ((imode & mode) != mode) return -1; 无 UID 检查 [3] kernel/syscall/sysproc.c · sys_getuid uint64 sys_getuid(void) { return 0; } 硬编码返回 0	[1] kernel/syscall/sysfile.c · sys_faccessat int sys_faccessat(void) { ... int imode = (ip->mode >> 6) & 0x7; if ((imode & mode) != mode) return -1; } // 仅检查模式位 [2] kernel/syscall/sysproc.c · sys_getuid uint64 sys_getuid(void) { return 0; } // 硬编码返回 0 [3] kernel/syscall/sysfile.c · sys_openat openat 实现中无 proc->uid 与 inode->uid 的比较逻辑

08_network — 网络子系统与协议栈

Q08_004 · short_answer

选择一个发送路径（优先 sys_sendto），追踪：syscall → 协议栈 → 网卡驱动。列 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	未发现 sys_sendto 实现。该内核未实现网络发送路径。poll/select 机制已就绪可用于未来 socket 集成：sys_pselect(kernel/syscall/sysfile.c:761) → pselect(kernel/fs/poll.c:127) → file_poll(kernel/fs/poll.c:58) → fp->poll 回调(如 pipepoll kernel/fs/pipe.c:378)	not_found: 该 OS 未实现网络子系统，无 sendto 系统调用，无协议栈，无网卡驱动。无法追踪发送路径。
证据	[1] kernel/syscall/sysfile.c · sys_pselect_impl sys_pselect:761 调用 pselect() 处理文件描述符多路复用；但无网络相关 syscall [2] kernel/fs/poll.c · pselect_impl pselect:127 遍历 fdset，调用 file_poll() 检查每个文件描述符的就绪状态 [3] kernel/fs/poll.c · file_poll_impl file_poll:58 检查 fp->poll 回调是否存在，若存在则调用 fp->poll(fp, pt) 返回掩码 [4] kernel/fs/pipe.c · pipepoll_impl pipepoll:378 实现 pipe 的 poll 回调，检查 pi->nwrite - pi->nread 判断是否有数据可读/可写	[1] include/sysnum.h · SYS_sendto 在 sysnum.h 中搜索 SYS_sendto，未找到该 syscall 定义。 [2] kernel/hal · hal drivers kernel/hal 目录包含 disk.c、sdcard.c、virtio_disk.c（块设备驱动）、spi.c、uart（串口）、gpiohs.c 等，但无 virtio_net.c、e1000.c、ne2000.c 等网卡驱动。 [3] kernel/hal/virtio.h · VIRTIO_BLK_T_virtio.h 中仅定义 VIRTIO_BLK_T_IN(0) 和 VIRTIO_BLK_T_OUT(1) 用于块设备读写，无 VIRTIO_NET_ 相关定义，说明 virtio 仅用于磁盘而非网络。

09_debug_error — 调试机制与错误处理

Q09_003 · short_answer

panic 路径会输出哪些诊断？（寄存器 dump/栈回溯/停机；必须引用实现证据）

项目	方法A	方法B
答案	panic 路径输出：1) 错误位置信息（文件路径、行号、hart ID）通过 panic 宏输出；2) 错误消息字符串；3) 栈回溯（backtrace）输出调用栈帧地址；4) 关闭中断后进入死循环停机。不包含寄存器 dump（trapframedump 仅在 trap 异常处理中调用，不在 panic 路径中）。证据：kernel/printf.c:123-132 __panic 函数调用 backtrace() 后 intr_off() 并死循环；kernel/trap/trap.c:129 trapframedump 仅在 usertrap 处理未知异常时调用。	Panic 路径会输出 panic 消息字符串、调用栈回溯（backtrace），然后关闭中断并进入无限循环停机。不直接输出通用寄存器 dump，但 backtrace 会打印帧中的返回地址。

项目	方法A	方法B
证据	[1] kernel/printf.c · __panic __panic 实现: printf 错误信息 → backtrace() → panicked=1 → intr_off() → for(;;); [2] kernel/trap/trap.c · trapframedump trapframedump (行 428-462) 输出所有通用寄存器 (a0-a7、t0-t6、s0-s11、ra、sp、gp、tp、epc), 但仅在 usertrap 未知异常路径调用 (行 129)	[1] kernel/printf.c · __panic void __panic(char s) {\n\tprintf(__ERROR("panic")):\n);\n\tprintf(s);\n\tprintf("\n");\n\tbacktrace();\n\tpanicked = 1; // freeze uart output from other CPUs\n\tintr_off();\n\tfor(;;) ;\n}\n输出消息, 调用 backtrace, 关中断, 死循环。 [2] kernel/printf.c · backtrace void backtrace() {\n\tuint64 fp = (uint64)r_fp();\n\tuint64 bottom = (uint64)PGROUNDUP((uint64)fp);\n\tprintf("backtrace:\n");\n\twhile (fp < bottom) {\n\t\tuint64 ra = (fp - 1);\n\t\tprintf("%p\n", ra - 4);\n\t\ttfp = (uint64)(fp - 2);\n\t\t}\n} 通过帧指针遍历栈帧, 打印返回地址 ra。

Q09_005 · tri_state_impl

是否存在 **内核驻留的交互式监视器 (kernel monitor)** ? (对齐 Stallings 《操作系统: 精髓与设计原理》语境: 在**内核态上下文**接受命令、用于探查/操控系统的监视器; **不包括**仅在用户态运行的常规 shell, 如 xv6-user/sh.c、user/ 下用户程序等——除非题面另有定义。必须三态; 若 implemented: 须给出 3-10 个 **用户可键入的 monitor 命令名** 及对应 **内核内** 解析/分发入口的 路径:行号 证据; 仅以用户态 shell 充当内核 monitor 视为 **未切题** 应判 stub 或 not_found 并说明理由。)

项目	方法A	方法B
答案	not_found	implemented
证据	[1] kernel/ · grep_monitor_gdbstub 在 kernel/目录搜索 monitor gdbstub gdb 关键字, 0 命中; 未发现内核态交互式命令解析循环或命令分发入口 [2] include/ · grep_monitor_command 在 include/目录搜索 monitor 相关头文件, 0 命中; 无内核 monitor 命令表或解析器定义	[1] kernel/console.c · consoleintr void consoleintr(int c) {\n\tacquire(&cons.lock);\n\tswitch(c){\n\tcase C('P'): // Print process list.\n\t\tprocdump();\n\t\tbreak;\n\tcase C('E'): // Print file-entry list.\n\t\trootfs_print();\n\t\tbreak;\n\tcase C('K'): // Print mem-alloc list.\n\t\tkmview();\n\t\tbreak;\n\tcase C('B'): // Print mem-alloc list.\n\t\tbprint();\n\t\tbreak;\n\tcase C('Q'): // Print physical memory status\n\t\ttpm_dump();\n\t\tbreak;\n\t}\n} 在内核控制台输入中断处理中解析特殊控制字符并执行内核诊断命令。 [2] kernel/console.c · monitor_commands 命令列表: Ctrl-P (procdump), Ctrl-E (rootfs_print), Ctrl-K (kmview), Ctrl-B (bprint), Ctrl-Q (pm_dump)。这些是在内核态 consoleintr 中直接处理的监视命令。

Q09_007 · short_answer

错误码/错误类型体系是什么? (errno/Result/Error enum; 给类型定义与传播点证据)

项目	方法A	方法B
答案	C 语言 errno 风格错误码体系。证据: include/errno.h:1-107 定义了标准 POSIX errno 宏 (EPERM、ENOENT、ENOMEM、EFAULT、EINVAL 等共 98+ 个错误码) ; kernel/mm/vm.c 等内核模块通过返回负值 (如 -EFAULT) 传播错误; 系统调用通过返回值传递 errno (负值表示错误) 。	使用标准的 errno 整数错误码体系。定义在 include/errno.h 中 (如 EPERM, ENOENT 等) 。系统调用通过返回负值的 errno 来传播错误 (如 -EBADF, -ENOMEM) 。

项目	方法A	方法B
证据	[1] include/errno.h · errno_definitions 定义 EPERM=1、ENOENT=2、ENOMEM=12、EFAULT=14、EINVAL=22、ENOSYS=38 等标准 POSIX 错误码 [2] kernel/console.c · consgetc 行 99 返回 -EFAULT 表示错误 [3] kernel/trap/trap.c · handle_excpc 行 405-425 返回 -1 表示不支持的异常，返回 0 表示成功处理	[1] include/errno.h · errno_definitions #define\tEPERM\t\t\t 1\t/ Operation not permitted /\n#define\tENOENT\t\t\t 2\t/ No such file or directory /\n... 定义了一系列标准 errno 错误码。 [2] kernel/syscall/sysfile.c · sys_read int sys_read(void) {\n\tstruct file *f;\n\tint n;\n\tuint64 p;\n\tif (argfd(0, 0, &f) < 0)\n\t\treturn -EBADF; // 返回负值 errno 表示错误\n... 系统调用返回负的错误码。 [3] kernel/syscall/sysproc.c · sys_sleep int sys_sleep(void) {\n...\n\tif (SIGTERM == p->killed) {\n\t\t\tret = -EINTR;\n\t\t\tbreak;\n\t}\n... 返回 -EINTR 表示被信号中断。

Q09_008 · tri_state_impl

是否存在 trace/perf/ftrace 等跟踪机制或 tracepoints？（必须三态）

项目	方法A	方法B
答案	stub	implemented
证据	[1] include/sysnum.h · SYS_trace 行 11 定义 SYS_trace=18 系统调用号，但仅为接口声明 [2] include/sched/proc.h · tmask 行 104 定义 struct proc 含 tmask 字段（trace mask），但仅为数据结构 [3] kernel/ · grep_tracepoint_ftrace 搜索 tracepoint ftrace perf 关键字，仅发现 proc_tms.utime 性能计数（行 171）和 SYS_trace 系统调用号，未发现 tracepoints 插入关键路径或 ftrace/perf 实现代码	[1] kernel/syscall/sysproc.c · sys_trace uint64 sys_trace(void) {\n\t// int mask;\n\t// if(argint(0, &mask) < 0) {\n\t\treturn -1;\n\t// }\n\tmyproc()->tmask = 1; // 设置进程跟踪掩码\n\treturn 0;\n} 实现 sys_trace 系统调用以启用跟踪。 [2] kernel/syscall/syscall.c · syscall void syscall(void) {\n...\n\t// trace\n\tint trace = p->tmask; // & (1 << (p->trapframe->a7 - 1));\n\tif (trace) {\n\t\t\tprintf("pid %d: %s(", p->pid, sysnames[num]);\n\t\t\t\n\t\t\ttrapframe->a0 = syscalls[num];\n\t\t\tif (trace) {\n\t\t\t\tprintf(") -> %d\n", p->trapframe->a0);\n\t\t\t\n\t\t\t... 根据 tmask 打印系统调用名称、参数和返回值，实现简单的 syscall trace。